

省选图图

Graph

QwQcOrZ Chinese_zjc_ Mine_King

2024 年 12 月 25 日

目录

- | | | |
|-----------|-------------|-------------------|
| 1. 虚树 | 5. 差分约束系统 | 10. 哈斯图 |
| 2. 各类联通分量 | 6. 欧拉图 | 11. 环计数问题 |
| 3. 圆方树 | 7. 竞赛图与哈密顿路 | 12. 最大团 / 最大独立集搜索 |
| 4. 2-SAT | 8. 弦图 | 13. 耳分解与双极定向 |
| | 9. 平面图 | 14. 广义串并联图 |

虚树

- 对于一棵树和一个关键点集合 S ，我们保留 S 中的关键点和任意两个关键点的 LCA，然后将树中的每条连接两个保留点的路径缩成一条边，这样得到了一棵新的树，我们称这棵新树为原树的虚树。

- 对于一棵树和一个关键点集合 S ，我们保留 S 中的关键点和任意两个关键点的 LCA，然后将树中的每条连接两个保留点的路径缩成一条边，这样得到了一棵新的树，我们称这棵新树为原树的虚树。
- 为了方便，有时候也会直接把根加入到虚树中。

- 对于一棵树和一个关键点集合 S ，我们保留 S 中的关键点和任意两个关键点的 LCA，然后将树中的每条连接两个保留点的路径缩成一条边，这样得到了一棵新的树，我们称这棵新树为原树的虚树。
- 为了方便，有时候也会直接把根加入到虚树中。
- 对于一些题目，如果我们只关心一些关键点，可以建出虚树然后缩小问题规模。

- 对于一棵树和一个关键点集合 S ，我们保留 S 中的关键点和任意两个关键点的 LCA，然后将树中的每条连接两个保留点的路径缩成一条边，这样得到了一棵新的树，我们称这棵新树为原树的虚树。
- 为了方便，有时候也会直接把根加入到虚树中。
- 对于一些题目，如果我们只关心一些关键点，可以建出虚树然后缩小问题规模。
- 根据 dfs 序的一些性质，显然虚树的大小至多为 $2|S| + 1$ 。

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。

构建

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。

构建

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。
- 初始时栈中只有一个点为根。

构建

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。
- 初始时栈中只有一个点为根。
- 每次加入一个关键点 u ，求出其与栈顶 v 的 LCA，然后将其与栈顶比较：

构建

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。
- 初始时栈中只有一个点为根。
- 每次加入一个关键点 u ，求出其与栈顶 v 的 LCA，然后将其与栈顶比较：
 - 如果 LCA 是栈顶，则直接将当前关键点压入栈中。

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。
- 初始时栈中只有一个点为根。
- 每次加入一个关键点 u ，求出其与栈顶 v 的 LCA，然后将其与栈顶比较：
 - 如果 LCA 是栈顶，则直接将当前关键点压入栈中。
 - 否则，弹出栈顶，然后考虑新的栈顶 w ，如果 w 是 LCA 的祖先，则需要将 LCA 压入栈中。

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。
- 初始时栈中只有一个点为根。
- 每次加入一个关键点 u ，求出其与栈顶 v 的 LCA，然后将其与栈顶比较：
 - 如果 LCA 是栈顶，则直接将当前关键点压入栈中。
 - 否则，弹出栈顶，然后考虑新的栈顶 w ，如果 w 是 LCA 的祖先，则需要将 LCA 压入栈中。
 - 将 v 的父亲设为此时的栈顶。

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。
- 初始时栈中只有一个点为根。
- 每次加入一个关键点 u ，求出其与栈顶 v 的 LCA，然后将其与栈顶比较：
 - 如果 LCA 是栈顶，则直接将当前关键点压入栈中。
 - 否则，弹出栈顶，然后考虑新的栈顶 w ，如果 w 是 LCA 的祖先，则需要将 LCA 压入栈中。
 - 将 v 的父亲设为此时的栈顶。
- 重复以上操作直到栈顶是 u 的祖先，然后将 u 压入栈中。

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。
- 初始时栈中只有一个点为根。
- 每次加入一个关键点 u ，求出其与栈顶 v 的 LCA，然后将其与栈顶比较：
 - 如果 LCA 是栈顶，则直接将当前关键点压入栈中。
 - 否则，弹出栈顶，然后考虑新的栈顶 w ，如果 w 是 LCA 的祖先，则需要将 LCA 压入栈中。
 - 将 v 的父亲设为此时的栈顶。
- 重复以上操作直到栈顶是 u 的祖先，然后将 u 压入栈中。
- 最后取出栈中所有节点，将每个点的父亲设为其下方的节点。

- 首先将关键点按 dfs 序排序，然后依次加入并在过程中维护虚树。
- 类似于建笛卡尔树，我们需要维护当前虚树的一条「右链」，将这条右链从上往下依次存在一个栈中。
- 初始时栈中只有一个点为根。
- 每次加入一个关键点 u ，求出其与栈顶 v 的 LCA，然后将其与栈顶比较：
 - 如果 LCA 是栈顶，则直接将当前关键点压入栈中。
 - 否则，弹出栈顶，然后考虑新的栈顶 w ，如果 w 是 LCA 的祖先，则需要将 LCA 压入栈中。
 - 将 v 的父亲设为此时的栈顶。
- 重复以上操作直到栈顶是 u 的祖先，然后将 u 压入栈中。
- 最后取出栈中所有节点，将每个点的父亲设为其下方的节点。
- 复杂度为 $O(T(\text{sort}) + |S|T(\text{LCA}))$ ，其中 $T(\text{sort})$ 和 $T(\text{LCA})$ 分别表示排序和求 LCA 的复杂度。

- 给一棵大小为 n 以 1 为根的树，每条边有边权， m 次询问，每次询问给出一些关键点，你要删除一些边，使得从 1 无法到达任意关键点，求删掉的边的边权和最小是多少。
- $2 \leq n \leq 2.5 \times 10^5, 1 \leq m \leq 5 \times 10^5$ ，总关键点数不超过 5×10^5 。

- 考虑单组询问怎么做，设 f_u 为只考虑 u 子树，从 u 出发无法到达任意非关键点的最小代价。

- 考虑单组询问怎么做，设 f_u 为只考虑 u 子树，从 u 出发无法到达任意非关键点的最小代价。

-

$$f_u = \sum_{v \text{ is a son of } u} \min\{f_v, w(u, v)\}$$

- 考虑单组询问怎么做，设 f_u 为只考虑 u 子树，从 u 出发无法到达任意非关键点的最小代价。

-

$$f_u = \sum_{v \text{ is a son of } u} \min\{f_v, w(u, v)\}$$

- 对于多组询问，考虑每次询问时只建出关键点的虚树，虚树的边权设为原树上对应路径的边权最小值即可。

各类联通分量

- 割点：删去这个点后使图不连通的点。

定义

- 割点：删去这个点后使图不连通的点。
- 割边：删去这条边后使图不连通的边。

- 割点：删去这个点后使图不连通的点。
- 割边：删去这条边后使图不连通的边。
- 点双连通分量：不存在割点的极大连通子图。

- 割点：删去这个点后使图不连通的点。
- 割边：删去这条边后使图不连通的边。
- 点双连通分量：不存在割点的极大连通子图。
- 边双连通分量：不存在割边的极大连通子图。

- 割点：删去这个点后使图不连通的点。
- 割边：删去这条边后使图不连通的边。
- 点双连通分量：不存在割点的极大连通子图。
- 边双连通分量：不存在割边的极大连通子图。
- 强连通分量：有向图中，点两两可达的极大连通子图。

- 一条边恰好在一个点双内，至多在一个边双内。

- 一条边恰好在一个点双内，至多在一个边双内。
- 一个点恰好在一个边双内，可以同时位于多个点双内。

- 一条边恰好在一个点双内，至多在一个边双内。
- 一个点恰好在一个边双内，可以同时位于多个点双内。
- 割边不在任何一个边双内。将所有割边删去后，剩下的每个连通块即为原图的所有边双。非割边恰好在一个边双内。

- 一条边恰好在一个点双内，至多在一个边双内。
- 一个点恰好在一个边双内，可以同时位于多个点双内。
- 割边不在任何一个边双内。将所有割边删去后，剩下的每个连通块即为原图的所有边双。非割边恰好在一个边双内。
- 有向图缩点之后是 DAG，边双缩点之后是树，点双可以考虑建立圆方树。

- 一条边恰好在一个点双内，至多在一个边双内。
- 一个点恰好在一个边双内，可以同时位于多个点双内。
- 割边不在任何一个边双内。将所有割边删去后，剩下的每个连通块即为原图的所有边双。非割边恰好在一个边双内。
- 有向图缩点之后是 DAG，边双缩点之后是树，点双可以考虑建立圆方树。
- 怎么求就不讲了，都是 tarjan 稍微改一改。

- Menger 定理有两种情况：

- Menger 定理有两种情况：
 1. 点连通性：一个非平凡图 G 是 k 点连通充分必要条件是：对于 G 的任意两个顶点 u, v , G 中存在 k 条点不相交的从 u 到 v 的路径。

- Menger 定理有两种情况：
 1. 点连通性：一个非平凡图 G 是 k 点连通充分必要条件是：对于 G 的任意两个顶点 u, v , G 中存在 k 条点不相交的从 u 到 v 的路径。
 2. 边连通性：一个非平凡图 G 是 k 边连通充分必要条件是：对于 G 的任意两个顶点 u, v , G 中存在 k 条边不相交的从 u 到 v 的路径。

- Menger 定理有两种情况：
 1. 点连通性：一个非平凡图 G 是 k 点连通充分必要条件是：对于 G 的任意两个顶点 u, v , G 中存在 k 条点不相交的从 u 到 v 的路径。
 2. 边连通性：一个非平凡图 G 是 k 边连通充分必要条件是：对于 G 的任意两个顶点 u, v , G 中存在 k 条边不相交的从 u 到 v 的路径。
- 本质就是最大流-最小割定理的特例，证明略。

- 题意和做法不用我多说了吧。



圆方树

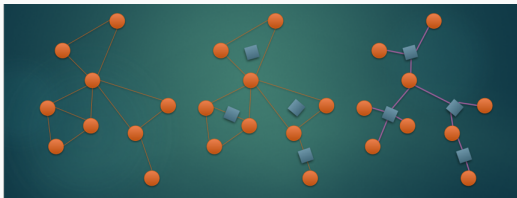
- 圆方树用于维护点双相关的信息，由圆点和方点构成。

圆方树

- 圆方树用于维护点双相关的信息，由圆点和方点构成。
- 圆点与原图中的点一一对应。

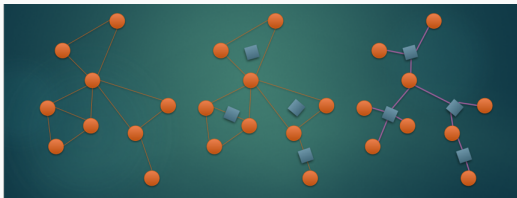
圆方树

- 圆方树用于维护点双相关的信息，由圆点和方点构成。
- 圆点与原图中的点一一对应。
- 对于每个点双建立一个方点，方点向所有点双内点对应的圆点连边，得到一个树形结构。



圆方树

- 圆方树用于维护点双相关的信息，由圆点和方点构成。
- 圆点与原图中的点一一对应。
- 对于每个点双建立一个方点，方点向所有点双内点对应的圆点连边，得到一个树形结构。



- 这样的图一定不存在环，否则可以将环上的方点合并为一个更大的点双，因此一定是一个树形结构。

圆方树的性质

- 任意两个方点、圆点不会相邻。

圆方树的性质

- 任意两个方点、圆点不会相邻。
- 所有叶子结点一定为圆点，非叶圆点为割点，叶子圆点为非割点。

圆方树的性质

- 任意两个方点、圆点不会相邻。
- 所有叶子结点一定为圆点，非叶圆点为割点，叶子圆点为非割点。
- 若两个方点距离为 2，则它们之间的圆点为这两个方点对应的点双的公共点。

圆方树的性质

- 任意两个方点、圆点不会相邻。
- 所有叶子结点一定为圆点，非叶圆点为割点，叶子圆点为非割点。
- 若两个方点距离为 2，则它们之间的圆点为这两个方点对应的点双的公共点。
- 对于两个圆点 a, b 之间的树上路径：

圆方树的性质

- 任意两个方点、圆点不会相邻。
- 所有叶子结点一定为圆点，非叶圆点为割点，叶子圆点为非割点。
- 若两个方点距离为 2，则它们之间的圆点为这两个方点对应的点双的公共点。
- 对于两个圆点 a, b 之间的树上路径：
 1. 路径上的圆点为必经点，即这 a, b 的割点。

圆方树的性质

- 任意两个方点、圆点不会相邻。
- 所有叶子结点一定为圆点，非叶圆点为割点，叶子圆点为非割点。
- 若两个方点距离为 2，则它们之间的圆点为这两个方点对应的点双的公共点。
- 对于两个圆点 a, b 之间的树上路径：
 1. 路径上的圆点为必经点，即这 a, b 的割点。
 2. a, b 所有简单路径并为路径上所有方点对应点双的并。

- 给出一个简单无向连通图，有 Q 次询问：
- 每次给出一个点集 S ，求有多少个点 u 满足 $u \notin S$ 且删去 u 后 S 中的点不连通。
- $T \leq 10, n \leq 10^5, m \leq 2 \times 10^5, Q \leq 10^5, \sum |S| \leq 2 \times 10^5$ 。

- 建出圆方树后问题转化为求 S 在圆方树上连通子图圆点个数减去 $|S|$ 。

- 建出圆方树后问题转化为求 S 在圆方树上连通子图圆点个数减去 $|S|$ 。
- 将圆点贡献放在其父边上，问题转化为边权和，加上一个特判 lca 处贡献。

- 建出圆方树后问题转化为求 S 在圆方树上连通子图圆点个数减去 $|S|$ 。
- 将圆点贡献放在其父边上，问题转化为边权和，加上一个特判 lca 处贡献。
- 将 $|S|$ 中点按 dfn 排序，计算每两个相邻的点，及首尾点的距离和，这个值的一半即为边权和。

- 给出一个简单无向图，求有多少三元组 (x, y, z) 满足：
 1. x, y, z 两两不相等。
 2. 存在一条从 x 到 z 的简单路径经过 y 。
- $n \leq 10^5, m \leq 2 \times 10^5$ 。

- 对于一个 x, z , 合法的 y 的集合为其路径上方点对应点双的并。

- 对于一个 x, z ，合法的 y 的集合为其路径上方点对应点双的并。
- 两个相邻方点之间的有公用圆点，因此考虑将方点权值记为其点双大小，圆点权值记为 -1 ，那么路径上权值和即为合法 y 的个数。

- 对于一个 x, z , 合法的 y 的集合为其路径上方点对应点双的并。
- 两个相邻方点之间的有公用圆点, 因此考虑将方点权值记为其点双大小, 圆点权值记为 -1 , 那么路径上权值和即为合法 y 的个数。
- 问题转化为所有两端是圆点的树上路径权值和, 计算每个点对答案的贡献即可。

- 给出一个简单无向图，每个点初始为黑色或白色。
- 对于每条边，需要选择是否将两端的点反色，求最后所有点都是白色的方案数。
- 另外，还需要对每个点求出删去这个点时问题的答案。
- $T \leq 5, n, m \leq 10^5$ ，答案对质数取模。

- 观察到每个连通块黑点个数奇偶性不会变，因此当有连通块黑点个数为奇数时答案一定为 0。

- 观察到每个连通块黑点个数奇偶性不会变，因此当有连通块黑点个数为奇数时答案一定为 0。
- 考虑树的情况，每条边是否需要反色是确定的，因此答案为 1。

- 观察到每个连通块黑点个数奇偶性不会变，因此当有连通块黑点个数为奇数时答案一定为 0。
- 考虑树的情况，每条边是否需要反色是确定的，因此答案为 1。
- 考虑连通图，撸出一棵生成树后，非树边随便选，树边策略固定，因此答案为 2^{m-n+1} 。

- 观察到每个连通块黑点个数奇偶性不会变，因此当有连通块黑点个数为奇数时答案一定为 0。
- 考虑树的情况，每条边是否需要反色是确定的，因此答案为 1。
- 考虑连通图，撸出一棵生成树后，非树边随便选，树边策略固定，因此答案为 2^{m-n+1} 。
- 考虑一般图，方案数为每个连通块的积，因此答案为 2^{m-n+b} ， b 为连通块数量。

- 观察到每个连通块黑点个数奇偶性不会变，因此当有连通块黑点个数为奇数时答案一定为 0。
- 考虑树的情况，每条边是否需要反色是确定的，因此答案为 1。
- 考虑连通图，撸出一棵生成树后，非树边随便选，树边策略固定，因此答案为 2^{m-n+1} 。
- 考虑一般图，方案数为每个连通块的积，因此答案为 2^{m-n+b} ， b 为连通块数量。
- 或者考虑异或方程组解数，答案为 2^t ， t 为自由元数量，可以得出同样的结论。

- 观察到每个连通块黑点个数奇偶性不会变，因此当有连通块黑点个数为奇数时答案一定为 0。
- 考虑树的情况，每条边是否需要反色是确定的，因此答案为 1。
- 考虑连通图，撸出一棵生成树后，非树边随便选，树边策略固定，因此答案为 2^{m-n+1} 。
- 考虑一般图，方案数为每个连通块的积，因此答案为 2^{m-n+b} ， b 为连通块数量。
- 或者考虑异或方程组解数，答案为 2^t ， t 为自由元数量，可以得出同样的结论。
- 建出圆方树后，计算每个点删除后 m, n, b 的值是简单的。

2-SAT

- SAT 是适定性 (Satisfiability) 问题的简称。一般形式被称为 k -SAT, 这里只讨论 2-SAT 问题。

- SAT 是适定性 (Satisfiability) 问题的简称。一般形式被称为 k-SAT, 这里只讨论 2-SAT 问题。
- 有 n 个 0/1 变量 x_i , 有若干个限制为 $(x_u/\neg x_u) \vee (x_v/\neg x_v)$ 为真。

- SAT 是适定性 (Satisfiability) 问题的简称。一般形式被称为 k-SAT, 这里只讨论 2-SAT 问题。
- 有 n 个 0/1 变量 x_i , 有若干个限制为 $(x_u/\neg x_u) \vee (x_v/\neg x_v)$ 为真。
- 求出一组合法解, 或者判定无解。

- 对于每个变量建两个点 i 和 i' 表示这个变量取 0/1。

- 对于每个变量建两个点 i 和 i' 表示这个变量取 0/1。
- 令一条有向边 (x, y) 的含义为由 x 能推出 y , 那么 $u \vee v$ 需要连边 (u', v) 和 (v', u) 。

解法一

- 考虑依次确定每个 x_i 的取值。

- 考虑依次确定每个 x_i 的取值。
- 先令 $x_i = 0$ ，然后 dfs 找到所有能推出的点，若某个 j 和 j' 均被推出，不合法，撤销继续检查 $x_i = 1$ 。

- 考虑依次确定每个 x_i 的取值。
- 先令 $x_i = 0$ ，然后 dfs 找到所有能推出的点，若某个 j 和 j' 均被推出，不合法，撤销继续检查 $x_i = 1$ 。
- 若找到合法解则继续做下一个 x_i ，若某个 x_i 均不合法则无解。

- 考虑依次确定每个 x_i 的取值。
- 先令 $x_i = 0$ ，然后 dfs 找到所有能推出的点，若某个 j 和 j' 均被推出，不合法，撤销继续检查 $x_i = 1$ 。
- 若找到合法解则继续做下一个 x_i ，若某个 x_i 均不合法则无解。
- 复杂度 $\mathcal{O}(n(n + m))$ ，但是可以用于求解字典序最小的解。

- 考虑对图进行缩 SCC。

解法二

- 考虑对图进行缩 SCC。
- 若 i 和 i' 在同一 SCC 内，无解。

- 考虑对图进行缩 SCC。
- 若 i 和 i' 在同一 SCC 内，无解。
- 否则选择 i 和 i' 中拓扑序较大的那个，这样一定不会推出另一个。

- 考虑对图进行缩 SCC。
- 若 i 和 i' 在同一 SCC 内，无解。
- 否则选择 i 和 i' 中拓扑序较大的那个，这样一定不会推出另一个。
- 复杂度 $\mathcal{O}(n + m)$ ，缺点是只能求出一组可行解。

- n 个标志要放在直线上，第 i 个标志可以放在坐标 x_i 或 y_i 上。
- 求相邻两个距离最小值的最大值。
- $n \leq 10^4, x_i, y_i \leq 10^9$ 。

- 二分答案之后问题变为 2-SAT 的问题，时间复杂度 $\mathcal{O}(n^2 \log n)$ 。

- 二分答案之后问题变为 2-SAT 的问题，时间复杂度 $\mathcal{O}(n^2 \log n)$ 。
- 连边是点向区间连边的形式，可以用线段树优化建图，时间复杂度 $\mathcal{O}(n \log^2 n)$ 。

- 有 n 个人， $T+1$ 个时刻， m 次预测，每条预测形如：
 1. 若 x 在时刻 t 已死亡， y 在时刻 $t+1$ 会死亡。
 2. 若 x 在时刻 t 仍存活， y 在时刻 t 会死亡。
- 预测一定正确，除此之外可能会发生自然死亡。人不会复活。
- 对每个人求出在 $T+1$ 时刻有多少人可能与其同时存活。
- $1 \leq t \leq T \leq 10^6, 1 \leq x, y \leq n \leq 5 \times 10^4, 1 \leq m \leq 10^5$ 。

- 建两种状态 $D(x, t)/A(x, t)$ 表示 x 在时刻 t 是否存活。

- 建两种状态 $D(x, t)/A(x, t)$ 表示 x 在时刻 t 是否存活。
- 首先有: $D(x, t) \rightarrow D(x, t+1), A(x, t) \rightarrow A(x, t-1)$ 。

- 建两种状态 $D(x, t)/A(x, t)$ 表示 x 在时刻 t 是否存活。
- 首先有: $D(x, t) \rightarrow D(x, t+1), A(x, t) \rightarrow A(x, t-1)$ 。
- 第一种限制: $D(x, t) \rightarrow D(y, t+1), A(y, t+1) \rightarrow A(x, t)$ 。

- 建两种状态 $D(x, t)/A(x, t)$ 表示 x 在时刻 t 是否存活。
- 首先有: $D(x, t) \rightarrow D(x, t+1), A(x, t) \rightarrow A(x, t-1)$ 。
- 第一种限制: $D(x, t) \rightarrow D(y, t+1), A(y, t+1) \rightarrow A(x, t)$ 。
- 第二种限制: $A(x, t) \rightarrow D(y, t), A(y, t) \rightarrow D(x, t)$ 。

- 建两种状态 $D(x, t)/A(x, t)$ 表示 x 在时刻 t 是否存活。
- 首先有: $D(x, t) \rightarrow D(x, t+1), A(x, t) \rightarrow A(x, t-1)$ 。
- 第一种限制: $D(x, t) \rightarrow D(y, t+1), A(y, t+1) \rightarrow A(x, t)$ 。
- 第二种限制: $A(x, t) \rightarrow D(y, t), A(y, t) \rightarrow D(x, t)$ 。
- 这样是 $\mathcal{O}(nT)$ 的, 但是可以只建出有用的状态, 复杂度 $\mathcal{O}(n+m)$ 。

- 建两种状态 $D(x, t)/A(x, t)$ 表示 x 在时刻 t 是否存活。
- 首先有: $D(x, t) \rightarrow D(x, t+1), A(x, t) \rightarrow A(x, t-1)$ 。
- 第一种限制: $D(x, t) \rightarrow D(y, t+1), A(y, t+1) \rightarrow A(x, t)$ 。
- 第二种限制: $A(x, t) \rightarrow D(y, t), A(y, t) \rightarrow D(x, t)$ 。
- 这样是 $\mathcal{O}(nT)$ 的, 但是可以只建出有用的状态, 复杂度 $\mathcal{O}(n+m)$ 。
- 对每个 $A(x, T+1)$ 求出可达的 $D(*, T+1)$ 的集合, 即可推出答案。

- 建两种状态 $D(x, t)/A(x, t)$ 表示 x 在时刻 t 是否存活。
- 首先有: $D(x, t) \rightarrow D(x, t+1), A(x, t) \rightarrow A(x, t-1)$ 。
- 第一种限制: $D(x, t) \rightarrow D(y, t+1), A(y, t+1) \rightarrow A(x, t)$ 。
- 第二种限制: $A(x, t) \rightarrow D(y, t), A(y, t) \rightarrow D(x, t)$ 。
- 这样是 $\mathcal{O}(nT)$ 的, 但是可以只建出有用的状态, 复杂度 $\mathcal{O}(n+m)$ 。
- 对每个 $A(x, T+1)$ 求出可达的 $D(*, T+1)$ 的集合, 即可推出答案。
- 直接 bitset 即可, 但是空间会爆, 需要分批处理。

- 求出公式 $Q_1x_1 Q_2x_2 \cdots Q_nx_n F(x_1, x_2, \cdots, x_n)$ 的值。
- 其中 $Q_i \in \{\forall, \exists\}$, F 是一个 2-SAT 形式的问题。
- $T, \sum n, \sum m \leq 10^5$

- 若 Q_i 全是 $\exists$, 那么就是一个 2-SAT 板子。

- 若 Q_i 全是 $\exists$ ，那么就是一个 2-SAT 板子。
- 对图缩 SCC，考虑什么情况会不合法：

- 若 Q_i 全是 $\exists$ ，那么就是一个 2-SAT 板子。
- 对图缩 SCC，考虑什么情况会不合法：
 1. 存在 x_i, x'_i 在同一 SCC 中。

- 若 Q_i 全是 $\exists$, 那么就是一个 2-SAT 板子。
- 对图缩 SCC, 考虑什么情况会不合法:
 1. 存在 x_i, x'_i 在同一 SCC 中。
 2. 存在 $i < j, Q_i = \exists, Q_j = \forall$, 且 x_i/x'_i 与 x_j/x'_j 在同一个 SCC 中 (Q_j 可以根据 Q_i 的取值来取对应不合法的值)。

- 若 Q_i 全是 $\exists$, 那么就是一个 2-SAT 板子。
- 对图缩 SCC, 考虑什么情况会不合法:
 1. 存在 x_i, x'_i 在同一 SCC 中。
 2. 存在 $i < j, Q_i = \exists, Q_j = \forall$, 且 x_i/x'_i 与 x_j/x'_j 在同一个 SCC 中 (Q_j 可以根据 Q_i 的取值来取对应不合法的值)。
 3. 存在 $i \neq j, Q_i = Q_j = \forall$, 且 x_i/x'_i 能到达 x_j/x'_j (同理)。

- 若 Q_i 全是 $\exists$, 那么就是一个 2-SAT 板子。
- 对图缩 SCC, 考虑什么情况会不合法:
 1. 存在 x_i, x'_i 在同一 SCC 中。
 2. 存在 $i < j, Q_i = \exists, Q_j = \forall$, 且 x_i/x'_i 与 x_j/x'_j 在同一个 SCC 中 (Q_j 可以根据 Q_i 的取值来取对应不合法的值)。
 3. 存在 $i \neq j, Q_i = Q_j = \forall$, 且 x_i/x'_i 能到达 x_j/x'_j (同理)。
- 必要性已经证明, 而且可以证明这些条件是充分的。

- 若 Q_i 全是 $\exists$ ，那么就是一个 2-SAT 板子。
- 对图缩 SCC，考虑什么情况会不合法：
 1. 存在 x_i, x'_i 在同一 SCC 中。
 2. 存在 $i < j, Q_i = \exists, Q_j = \forall$ ，且 x_i/x'_i 与 x_j/x'_j 在同一个 SCC 中（ Q_j 可以根据 Q_i 的取值来取对应不合法的值）。
 3. 存在 $i \neq j, Q_i = Q_j = \forall$ ，且 x_i/x'_i 能到达 x_j/x'_j （同理）。
- 必要性已经证明，而且可以证明这些条件是充分的。
- 考虑构造证明：所有与 $\forall$ 在同一个 SCC 中的点由 $\forall$ 的取值来确定取值，能推出不选的 $\forall$ 的点全部不能选，选的 $\forall$ 能推出的点全部必须选。其它点跑 2-SAT 即可。

- 若 Q_i 全是 $\exists$ ，那么就是一个 2-SAT 板子。
- 对图缩 SCC，考虑什么情况会不合法：
 1. 存在 x_i, x'_i 在同一 SCC 中。
 2. 存在 $i < j, Q_i = \exists, Q_j = \forall$ ，且 x_i/x'_i 与 x_j/x'_j 在同一个 SCC 中（ Q_j 可以根据 Q_i 的取值来取对应不合法的值）。
 3. 存在 $i \neq j, Q_i = Q_j = \forall$ ，且 x_i/x'_i 能到达 x_j/x'_j （同理）。
- 必要性已经证明，而且可以证明这些条件是充分的。
- 考虑构造证明：所有与 $\forall$ 在同一个 SCC 中的点由 $\forall$ 的取值来确定取值，能推出不选的 $\forall$ 的点全部不能选，选的 $\forall$ 能推出的点全部必须选。其它点跑 2-SAT 即可。
- 若过程中推出矛盾，则一定会出现上面三种情况之一。

差分约束系统

- 差分约束系统是一种特殊的 n 元一次不等式组。

- 差分约束系统是一种特殊的 n 元一次不等式组。
- 问题的形式为 n 个变量 $x_1, x_2, \dots, x_n$, m 个约束形如 $x_i - x_j \leq c_k$ 。

- 差分约束系统是一种特殊的 n 元一次不等式组。
- 问题的形式为 n 个变量 $x_1, x_2, \dots, x_n$, m 个约束形如 $x_i - x_j \leq c_k$ 。
- 要求求出一组合法解或判定无解。

- 将 $x_i - x_j \leq c_k$ 移项得到 $x_i \leq x_j + c_k$ 。

- 将 $x_i - x_j \leq c_k$ 移项得到 $x_i \leq x_j + c_k$ 。
- 由于这与最短路三角不等式类似，考虑用一条边 (j, i, c_k) 来表示。

- 将 $x_i - x_j \leq c_k$ 移项得到 $x_i \leq x_j + c_k$ 。
- 由于这与最短路三角不等式类似，考虑用一条边 (j, i, c_k) 来表示。
- 当图不连通时，可以建立虚点 0，对于每个 i 连一条边 $(0, i, 0)$ 。这显然不会影响系统的有解性。

- 将 $x_i - x_j \leq c_k$ 移项得到 $x_i \leq x_j + c_k$ 。
- 由于这与最短路三角不等式类似，考虑用一条边 (j, i, c_k) 来表示。
- 当图不连通时，可以建立虚点 0，对于每个 i 连一条边 $(0, i, 0)$ 。这显然不会影响系统的有解性。
- 从 0 开始跑 spfa，求出的最短路即为一组可行解（因为最短路满足三角不等式）。否则可以判定出图中存在负环，无解。

- 将 $x_i - x_j \leq c_k$ 移项得到 $x_i \leq x_j + c_k$ 。
- 由于这与最短路三角不等式类似，考虑用一条边 (j, i, c_k) 来表示。
- 当图不连通时，可以建立虚点 0，对于每个 i 连一条边 $(0, i, 0)$ 。这显然不会影响系统的有解性。
- 从 0 开始跑 spfa，求出的最短路即为一组可行解（因为最短路满足三角不等式）。否则可以判定出图中存在负环，无解。
- 正确性：若存在负环 $p_1 \rightarrow p_2 \rightarrow \cdots \rightarrow p_n \rightarrow p_1$ ，则可以推出 $x_{p_1} \leq x_{p_1} + C$ ，其中 C 为负环的边权和。不存在可行解。

- 将 $x_i - x_j \leq c_k$ 移项得到 $x_i \leq x_j + c_k$ 。
- 由于这与最短路三角不等式类似，考虑用一条边 (j, i, c_k) 来表示。
- 当图不连通时，可以建立虚点 0，对于每个 i 连一条边 $(0, i, 0)$ 。这显然不会影响系统的有解性。
- 从 0 开始跑 spfa，求出的最短路即为一组可行解（因为最短路满足三角不等式）。否则可以判定出图中存在负环，无解。
- 正确性：若存在负环 $p_1 \rightarrow p_2 \rightarrow \cdots \rightarrow p_n \rightarrow p_1$ ，则可以推出 $x_{p_1} \leq x_{p_1} + C$ ，其中 C 为负环的边权和。不存在可行解。
- 复杂度 $\mathcal{O}(\text{spfa}(n, m))$ 。

- 将 $x_i - x_j \leq c_k$ 移项得到 $x_i \leq x_j + c_k$ 。
- 由于这与最短路三角不等式类似，考虑用一条边 (j, i, c_k) 来表示。
- 当图不连通时，可以建立虚点 0，对于每个 i 连一条边 $(0, i, 0)$ 。这显然不会影响系统的有解性。
- 从 0 开始跑 spfa，求出的最短路即为一组可行解（因为最短路满足三角不等式）。否则可以判定出图中存在负环，无解。
- 正确性：若存在负环 $p_1 \rightarrow p_2 \rightarrow \cdots \rightarrow p_n \rightarrow p_1$ ，则可以推出 $x_{p_1} \leq x_{p_1} + C$ ，其中 C 为负环的边权和。不存在可行解。
- 复杂度 $\mathcal{O}(\text{spfa}(n, m))$ 。
- 注意判负环的时候一般情况下不要真的判定入队次数是否达到 n ，而是设立一个尽量大不会 TLE 的上界。

- 有一个 $n \times m$ 的矩阵 A ，给出一个 $(n-1) \times (m-1)$ 的矩形 B ，满足 $b_{i,j} = a_{i,j} + a_{i+1,j} + a_{i,j+1} + a_{i+1,j+1}$ 。
- 请构造一个满足条件的 A 或者判定无解，要求 $0 \leq a_{i,j} \leq 10^6$ 。
- $T \leq 10, 2 \leq n, m \leq 300, 0 \leq b_{i,j} \leq 4 \times 10^6$ 。

- 考虑先搞出任意一个合法的 A (不考虑范围限制), 然后进行一些调整使其满足范围。

- 考虑先搞出任意一个合法的 A (不考虑范围限制), 然后进行一些调整使其满足范围。
- 对每列进行这样的调整:

$$\begin{bmatrix} +c_1 & -c_2 & +c_3 & -c_4 \\ -c_1 & +c_2 & -c_3 & +c_4 \\ +c_1 & -c_2 & +c_3 & -c_4 \\ -c_1 & +c_2 & -c_3 & +c_4 \end{bmatrix} \quad (1)$$

- 考虑先搞出任意一个合法的 A (不考虑范围限制), 然后进行一些调整使其满足范围。
- 对每列进行这样的调整:

$$\begin{bmatrix} +c_1 & -c_2 & +c_3 & -c_4 \\ -c_1 & +c_2 & -c_3 & +c_4 \\ +c_1 & -c_2 & +c_3 & -c_4 \\ -c_1 & +c_2 & -c_3 & +c_4 \end{bmatrix} \quad (1)$$

- 对每行进行这样的调整:

$$\begin{bmatrix} -d_1 & +d_1 & -d_1 & +d_1 \\ +d_2 & -d_2 & +d_2 & -d_2 \\ -d_3 & +d_3 & -d_3 & +d_3 \\ +d_4 & -d_4 & +d_4 & -d_4 \end{bmatrix} \quad (2)$$

- 对于每个位置，可以列出不等式 $0 \leq a_{i,j} \pm (c_j - d_i) \leq 10^6$ 。使用差分约束寻找解即可。

- 对于每个位置，可以列出不等式 $0 \leq a_{i,j} \pm (c_j - d_i) \leq 10^6$ 。使用差分约束寻找解即可。
- 考虑正确性，首先如果确定了 A 的第一行第一列，那么整个矩阵都可以被唯一推出。

- 对于每个位置，可以列出不等式 $0 \leq a_{i,j} \pm (c_j - d_i) \leq 10^6$ 。使用差分约束寻找解即可。
- 考虑正确性，首先如果确定了 A 的第一行第一列，那么整个矩阵都可以被唯一推出。
- 那么上述调整方式显然能遍历所有第一行第一列的情况，因此有解一定被找到。

- 给出一张连通混合图，求出是否存在一组点权 $x_1, x_2, \dots, x_n$ ，使得对于所有有向边 (u, v) 满足 $x_v = x_u + 1$ ，无向边 (u, v) 满足 $|x_v - x_u| = 1$ 。
- 若存在解，则构造一组 x 的极差最大的解。
- $n \leq 200, m \leq 2000$ 。

- 先解决第一问：求出是否存在一组满足条件的解。

- 先解决第一问：求出是否存在一组满足条件的解。
- 对于有向边 (u, v) ，连边 $(u, v, 1), (v, u, -1)$ 。

- 先解决第一问：求出是否存在一组满足条件的解。
- 对于有向边 (u, v) ，连边 $(u, v, 1), (v, u, -1)$ 。
- 对于无向边 (u, v) ，连边 $(u, v, 1), (v, u, 1)$ 。

- 先解决第一问：求出是否存在一组满足条件的解。
- 对于有向边 (u, v) ，连边 $(u, v, 1), (v, u, -1)$ 。
- 对于无向边 (u, v) ，连边 $(u, v, 1), (v, u, 1)$ 。
- 无向边的限制与题目并不完全等价，但是由于最短路性质，若有解一定能求出一组合法解。

- 先解决第一问：求出是否存在一组满足条件的解。
- 对于有向边 (u, v) ，连边 $(u, v, 1), (v, u, -1)$ 。
- 对于无向边 (u, v) ，连边 $(u, v, 1), (v, u, 1)$ 。
- 无向边的限制与题目并不完全等价，但是由于最短路性质，若有解一定能求出一组合法解。
- 证明：相邻的点奇偶性一定不同，那么若存在奇环一定不合法。否则求出的解一定满足其奇偶性，否则可以找到更短的最短路。

- 先解决第一问：求出是否存在一组满足条件的解。
- 对于有向边 (u, v) ，连边 $(u, v, 1), (v, u, -1)$ 。
- 对于无向边 (u, v) ，连边 $(u, v, 1), (v, u, 1)$ 。
- 无向边的限制与题目并不完全等价，但是由于最短路的性质，若有解一定能求出一组合法解。
- 证明：相邻的点奇偶性一定不同，那么若存在奇环一定不合法。否则求出的解一定满足其奇偶性，否则可以找到更短的最短路。
- 至于第二问，将最短路改为 Floyd，求出每个点为起点的最短路即可。极差显然不可能超过最短路。

欧拉图

- 欧拉路：经过每条边恰好一次的路径。

- 欧拉路：经过每条边恰好一次的路径。
- 欧拉回路：经过每条边恰好一次的回路。

- 欧拉路：经过每条边恰好一次的路径。
- 欧拉回路：经过每条边恰好一次的回路。
- 半欧拉图：存在欧拉路但不存在欧拉回路的图。

- 欧拉路：经过每条边恰好一次的路径。
- 欧拉回路：经过每条边恰好一次的回路。
- 半欧拉图：存在欧拉路但不存在欧拉回路的图。
- 欧拉图：存在欧拉回路的图。

- 无向图：

- 无向图：
 1. 欧拉图的判定：连通且所有点度数为偶数。

- 无向图：
 1. 欧拉图的判定：连通且所有点度数为偶数。
 2. 半欧拉图的判定：连通且奇点数量为 2，这两个奇点会成为起点和终点。

- 无向图：
 1. 欧拉图的判定：连通且所有点度数为偶数。
 2. 半欧拉图的判定：连通且奇点数量为 2，这两个奇点会成为起点和终点。
- 有向图：

- 无向图：
 1. 欧拉图的判定：连通且所有点度数为偶数。
 2. 半欧拉图的判定：连通且奇点数量为 2，这两个奇点会成为起点和终点。
- 有向图：
 1. 欧拉图的判定：弱连通且所有点入度等于出度。

- 无向图：

1. 欧拉图的判定：连通且所有点度数为偶数。
2. 半欧拉图的判定：连通且奇点数量为 2，这两个奇点会成为起点和终点。

- 有向图：

1. 欧拉图的判定：弱连通且所有点入度等于出度。
2. 半欧拉图的判定：弱连通且恰有一个点 u 入度比出度小 1，一个点 v 入度比出度大 1，其余顶点入度等于出度，此时 u, v 分别成为起点和终点。

- 无向图：

1. 欧拉图的判定：连通且所有点度数为偶数。
2. 半欧拉图的判定：连通且奇点数量为 2，这两个奇点会成为起点和终点。

- 有向图：

1. 欧拉图的判定：弱连通且所有点入度等于出度。
2. 半欧拉图的判定：弱连通且恰有一个点 u 入度比出度小 1，一个点 v 入度比出度大 1，其余顶点入度等于出度，此时 u, v 分别成为起点和终点。
3. 推论：如果图弱连通且所有点入度与出度差的绝对值之和为 $2k$ ，那么可以用 k 条路径将图的每条边经过一次。

- 用于求解图的欧拉（回）路。

- 用于求解图的欧拉（回）路。
- 若直接 dfs 求解，走向一个所有边都用过的点可能没走完整个图，所以得倒回去走环。

- 用于求解图的欧拉（回）路。
- 若直接 dfs 求解，走完一个所有边都用过的点可能没走完整个图，所以得倒回去走环。
- 在 dfs 时，倒着把边加到答案里即可。

- 用于求解图的欧拉（回）路。
- 若直接 dfs 求解，走完一个所有边都用过的点可能没走完整个图，所以得倒回去走环。
- 在 dfs 时，倒着把边加到答案里即可。
- 半欧拉图上只要选好起点就可以了。

- 用于求解图的欧拉（回）路。
- 若直接 dfs 求解，走完一个所有边都用过的点可能没走完整个图，所以得倒回去走环。
- 在 dfs 时，倒着把边加到答案里即可。
- 半欧拉图上只要选好起点就可以了。
- 找字典序最小的欧拉（回）路：每次贪心即正确。

- 一个长度为 $nm + 1$ 的序列 a , $[1, m]$ 中的每个整数恰好出现了 n 次, 最后一个是 0。
- 每次操作可以将一个数与 0 交换, 求出一个长度最短的操作序列使得序列每 m 个元素分一段后每段都是一个排列。
- $n, m \leq 400$ 。

- 每种数的具体位置是无关紧要的，只关心其位于哪一段。

- 每种数的具体位置是无关紧要的，只关心其位于哪一段。
- 对于每段，将每种出现了的数保留一个，显然我们不会去动它。

- 每种数的具体位置是无关紧要的，只关心其位于哪一段。
- 对于每段，将每种出现了的数保留一个，显然我们不会去动它。
- 若第 i 段有多余的第 j 种元素，则连边 $(j + n, i)$ 共多余个数条。

- 每种数的具体位置是无关紧要的，只关心其位于哪一段。
- 对于每段，将每种出现了的数保留一个，显然我们不会去动它。
- 若第 i 段有多余的第 j 种元素，则连边 $(j + n, i)$ 共多余个数条。
- 若第 i 段没有第 j 种元素，则连边 $(i, j + n)$ 。

- 每种数的具体位置是无关紧要的，只关心其位于哪一段。
- 对于每段，将每种出现了的数保留一个，显然我们不会去动它。
- 若第 i 段有多余的第 j 种元素，则连边 $(j + n, i)$ 共多余个数条。
- 若第 i 段没有第 j 种元素，则连边 $(i, j + n)$ 。
- 对每个连通块跑欧拉回路即可，每走一次 $(i, j + n)$ 表示移动一个 j 到当前空位，每走一次 $(j + n, i)$ 表示要移动的数位于第 i 段。

- n 个长度为不超过 2 的串，把它们以任意顺序拼接起来，每个串可以正着放也可以反着放。
- 构造一种方案使得拼出来是回文串，保证有解。
- $n, m \leq 5 \times 10^5$, m 为字符集大小。

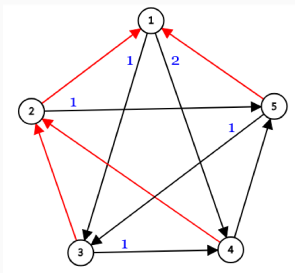
- 描述起来有点麻烦，直接看[官方题解](#)吧。

有向欧拉图欧拉回路计数 (BEST 定理)

- 假设 1 为起点, 考虑构造双射: 找到一棵以 1 为根的内向生成树, 对于每个点的非树边出边钦定一个顺序。上述方案与欧拉回路一一对应。

有向欧拉图欧拉回路计数 (BEST 定理)

- 假设 1 为起点，考虑构造双射：找到一棵以 1 为根的内向生成树，对于每个点的非树边出边钦定一个顺序。上述方案与欧拉回路一一对应。
- 一个构造方案的例子：



有向欧拉图欧拉回路计数 (BEST 定理)

- 首先证明一个方案唯一对应一条回路：从 1 出发，若还有非树边则按顺序走非树边，否则走树边。

有向欧拉图欧拉回路计数 (BEST 定理)

- 首先证明一个方案唯一对应一条回路：从 1 出发，若还有非树边则按顺序走非树边，否则走树边。
- 现在只要证明每次走完边剩下的图仍然存在从其出发的欧拉路径，即可证明这条路径就是欧拉路径。

有向欧拉图欧拉回路计数 (BEST 定理)

- 首先证明一个方案唯一对应一条回路：从 1 出发，若还有非树边则按顺序走非树边，否则走树边。
- 现在只要证明每次走完边剩下的图仍然存在从其出发的欧拉路径，即可证明这条路径就是欧拉路径。
- 根据欧拉路径的判定，可以发现这个条件等价于每次走完非树边 (u, v) 后 u, v 弱连通。

有向欧拉图欧拉回路计数 (BEST 定理)

- 首先证明一个方案唯一对应一条回路：从 1 出发，若还有非树边则按顺序走非树边，否则走树边。
- 现在只要证明每次走完边剩下的图仍然存在从其出发的欧拉路径，即可证明这条路径就是欧拉路径。
- 根据欧拉路径的判定，可以发现这个条件等价于每次走完非树边 (u, v) 后 u, v 弱连通。
- 当我们经过一条非树边 (u, v) 时， u, v 的树边一定存在，它们父亲的树边也一定存在。依次类推，得到它们都与根弱连通。

有向欧拉图欧拉回路计数 (BEST 定理)

- 首先证明一个方案唯一对应一条回路：从 1 出发，若还有非树边则按顺序走非树边，否则走树边。
- 现在只要证明每次走完边剩下的图仍然存在从其出发的欧拉路径，即可证明这条路径就是欧拉路径。
- 根据欧拉路径的判定，可以发现这个条件等价于每次走完非树边 (u, v) 后 u, v 弱连通。
- 当我们经过一条非树边 (u, v) 时， u, v 的树边一定存在，它们父亲的树边也一定存在。依次类推，得到它们都与根弱连通。
- 原因是原图是欧拉图，每个点入度等于出度，走完当前路径后的图满足起点入度比出度大 1，终点出度比入度大 1，其余结点入度等于出度。

有向欧拉图欧拉回路计数 (BEST 定理)

- 再证明一条欧拉回路唯一对应一个方案：对于一条以 1 号点为起点和终点的欧拉回路，除 1 号点外的每个点最后访问的出边是树边，其余出边按照访问次序确定顺序。

有向欧拉图欧拉回路计数 (BEST 定理)

- 再证明一条欧拉回路唯一对应一个方案：对于一条以 1 号点为起点和终点的欧拉回路，除 1 号点外的每个点最后访问的出边是树边，其余出边按照访问次序确定顺序。
- 接下来需要证明树边不会形成环。

有向欧拉图欧拉回路计数 (BEST 定理)

- 再证明一条欧拉回路唯一对应一个方案：对于一条以 1 号点为起点和终点的欧拉回路，除 1 号点外的每个点最后访问的出边是树边，其余出边按照访问次序确定顺序。
- 接下来需要证明树边不会形成环。
- 考虑反证：假设树边形成了环，由于 1 号点不选择树边因此其一定不在环上。从环上任意一点出发，绕一圈回到这个点，由于树边是最后被访问的一条边，所以欧拉路径终止于此点，这与欧拉路径的终点为 1 矛盾。

有向欧拉图欧拉回路计数 (BEST 定理)

- 再证明一条欧拉回路唯一对应一个方案：对于一条以 1 号点为起点和终点的欧拉回路，除 1 号点外的每个点最后访问的出边是树边，其余出边按照访问次序确定顺序。
- 接下来需要证明树边不会形成环。
- 考虑反证：假设树边形成了环，由于 1 号点不选择树边因此其一定不在环上。从环上任意一点出发，绕一圈回到这个点，由于树边是最后被访问的一条边，所以欧拉路径终止于此点，这与欧拉路径的终点为 1 矛盾。
- 至此，我们证明了方案和欧拉路径形成了双射关系。

有向欧拉图欧拉回路计数 (BEST 定理)

- 考虑计数, 记 T_i 为以 i 为根的内向树数量, d_i 为 i 号点的出度。

有向欧拉图欧拉回路计数 (BEST 定理)

- 考虑计数, 记 T_i 为以 i 为根的内向树数量, d_i 为 i 号点的出度。
- 对于一棵内向树, 1 号点非树出边有 d_1 条, 其余点非树出边有 $d_i - 1$ 条, 要给这些边定顺序, 得到方案数为:

$$T_1 d_1! \prod_{i \neq 1} (d_i - 1)! \quad (3)$$

有向欧拉图欧拉回路计数 (BEST 定理)

- 考虑计数, 记 T_i 为以 i 为根的内向树数量, d_i 为 i 号点的出度。
- 对于一棵内向树, 1 号点非树出边有 d_1 条, 其余点非树出边有 $d_i - 1$ 条, 要给这些边定顺序, 得到方案数为:

$$T_1 d_1! \prod_{i \neq 1} (d_i - 1)! \quad (3)$$

- T_1 可以通过矩阵树定理 $\mathcal{O}(n^3)$ 求出。

有向欧拉图欧拉回路计数 (BEST 定理)

- 为了避免重复计算循环同构的欧拉回路，我们需要将一个欧拉回路转化为唯一的一条欧拉路径。

有向欧拉图欧拉回路计数 (BEST 定理)

- 为了避免重复计算循环同构的欧拉回路，我们需要将一个欧拉回路转化为唯一的一条欧拉路径。
- 规定从 1 号点出发的标号最小的边作为第一条访问边，我们就能够得到不同的欧拉回路的数量：

$$T_1 \prod (d_i - 1)! \quad (4)$$

有向欧拉图欧拉回路计数 (BEST 定理)

- 为了避免重复计算循环同构的欧拉回路，我们需要将一个欧拉回路转化为唯一的一条欧拉路径。
- 规定从 1 号点出发的标号最小的边作为第一条访问边，我们就能够得到不同的欧拉回路的数量：

$$T_1 \prod (d_i - 1)! \quad (4)$$

- 这一数量关系由 de Bruijn、van Aardenne-Ehrenfest、Smith 和 Tutte 四人提出，称为 BEST 定理。

有向欧拉图欧拉回路计数 (BEST 定理)

- 为了避免重复计算循环同构的欧拉回路，我们需要将一个欧拉回路转化为唯一的一条欧拉路径。
- 规定从 1 号点出发的标号最小的边作为第一条访问边，我们就能够得到不同的欧拉回路的数量：

$$T_1 \prod (d_i - 1)! \quad (4)$$

- 这一数量关系由 de Bruijn、van Aardenne-Ehrenfest、Smith 和 Tutte 四人提出，称为 BEST 定理。
- 从这个公式中，还可以看出有向欧拉图具有 $T_1 = T_2 = \cdots = T_n$ 的性质。

- 通过加一条边将半欧拉图 G 变成欧拉图 G' ，可以发现 G' 上的欧拉回路与 G 上的欧拉路径一一对应。

有向半欧拉图欧拉路径计数

- 通过加一条边将半欧拉图 G 变成欧拉图 G' ，可以发现 G' 上的欧拉回路与 G 上的欧拉路径一一对应。
- 使用 BEST 定理求解即可。

竞赛图与哈密顿路

- 哈密顿通路：经过图中所有顶点恰好一次的路径。

- 哈密顿通路：经过图中所有顶点恰好一次的路径。
- 哈密顿回路：经过图中所有顶点恰好一次的回路。

- 哈密顿通路：经过图中所有顶点恰好一次的路径。
- 哈密顿回路：经过图中所有顶点恰好一次的回路。
- 哈密顿图：存在哈密顿回路的图。

- 哈密顿通路：经过图中所有顶点恰好一次的路径。
- 哈密顿回路：经过图中所有顶点恰好一次的回路。
- 哈密顿图：存在哈密顿回路的图。
- 半哈密顿图：存在哈密顿通路但不存在哈密顿回路的图。

- 哈密顿通路：经过图中所有顶点恰好一次的路径。
- 哈密顿回路：经过图中所有顶点恰好一次的回路。
- 哈密顿图：存在哈密顿回路的图。
- 半哈密顿图：存在哈密顿通路但不存在哈密顿回路的图。
- 竞赛图：任意两点之间恰好有一条有向边的有向图。

- 竞赛图缩点后是一张完全 DAG，或者有时可以直接看成链。

- 竞赛图缩点后是一张完全 DAG，或者有时可以直接看成链。
- 若一个竞赛图中存在环，则该图中一定存在三元环。

- 竞赛图缩点后是一张完全 DAG，或者有时可以直接看成链。
- 若一个竞赛图中存在环，则该图中一定存在三元环。
- 证明：考虑反证，记 $c_1, c_2, \dots, c_k (k > 3)$ 为图中的最小环，讨论 c_1 与 c_3 之间的连边方向，我们一定能找到一个更小的环。

- 一般图哈密顿回路的存在性判断是 NPC 的。

- 一般图哈密顿回路的存在性判断是 NPC 的。
- 竞赛图一定存在哈密顿通路。

- 一般图哈密顿回路的存在性判断是 NPC 的。
- 竞赛图一定存在哈密顿通路。
- 证明：考虑每次加入一个点 x ，一定满足以下条件之一：

- 一般图哈密顿回路的存在性判断是 NPC 的。
- 竞赛图一定存在哈密顿通路。
- 证明：考虑每次加入一个点 x ，一定满足以下条件之一：
 1. x 连向起点。

- 一般图哈密顿回路的存在性判断是 NPC 的。
- 竞赛图一定存在哈密顿通路。
- 证明：考虑每次加入一个点 x ，一定满足以下条件之一：
 1. x 连向起点。
 2. 终点连向 x 。

- 一般图哈密顿回路的存在性判断是 NPC 的。
- 竞赛图一定存在哈密顿通路。
- 证明：考虑每次加入一个点 x ，一定满足以下条件之一：
 1. x 连向起点。
 2. 终点连向 x 。
 3. 否则一定存在一个位置 i 满足同时存在边 $(i, x), (x, i+1)$ ，将其插入即可。

- 强连通竞赛图一定存在哈密顿回路。

- 强连通竞赛图一定存在哈密顿回路。
- 构造方法：先找出一条哈密顿通路，找到最后一个连向起点的点，构成一个回路。

- 强连通竞赛图一定存在哈密顿回路。
- 构造方法：先找出一条哈密顿通路，找到最后一个连向起点的点，构成一个回路。
- 记当前回路起点为 S ，回路上连向 S 的点为 T ， T 在通路上连向的点为 u 。

- 强连通竞赛图一定存在哈密顿回路。
- 构造方法：先找出一条哈密顿通路，找到最后一个连向起点的点，构成一个回路。
- 记当前回路起点为 S ，回路上连向 S 的点为 T ， T 在通路上连向的点为 u 。
- 每次找到通路上第一个有连向回路上的边的点 i （由于强连通其一定存在），记其连向的最靠前的点为 x ， x 在回路上的前驱为 y ，构造回路： $S \rightarrow y \rightarrow u \rightarrow i \rightarrow x \rightarrow T \rightarrow S$ 。

- 强连通竞赛图一定存在哈密顿回路。
- 构造方法：先找出一条哈密顿通路，找到最后一个连向起点的点，构成一个回路。
- 记当前回路起点为 S ，回路上连向 S 的点为 T ， T 在通路上连向的点为 u 。
- 每次找到通路上第一个有连向回路上的边的点 i （由于强连通其一定存在），记其连向的最靠前的点为 x ， x 在回路上的前驱为 y ，构造回路： $S \rightarrow y \rightarrow u \rightarrow i \rightarrow x \rightarrow T \rightarrow S$ 。
- 不断重复此过程直到所有点都加入回路即可。

- 给定一张竞赛图，求从每个点开始的最长路径，并输出长度与方案。
- $n \leq 2 \times 10^3$ 。

- 缩点，对每个 SCC 找一条哈密顿回路，每个点的答案为在当前 SCC 内绕一圈拼上后面 SCC 的答案。

- 给定一张完全图，现在要给每条边定向，其中 k 条特殊边 (u_i, v_i) 定向的概率为 p_i ，其余边的定向概率都为 $\frac{1}{2}$ 。
- 求这张图的期望强连通分类数，对 998244353 取模。
- $n \leq 38, k \leq 19$ 。

- 由于竞赛图缩点之后是链，答案为这条链的期望点数。考虑枚举链上的每一条边，每条边都会对答案产生 1 的贡献。

- 由于竞赛图缩点之后是链，答案为这条链的期望点数。考虑枚举链上的每一条边，每条边都会对答案产生 1 的贡献。
- 若在这条边之前的点集 S ，那么这条边存在的概率为：

$$\left(\frac{1}{2}\right)^{|S|(n-|S|)} \times \prod_{u_i \in S, v_i \notin S} 2p_i \quad (5)$$

- 由于竞赛图缩点之后是链，答案为这条链的期望点数。考虑枚举链上的每一条边，每条边都会对答案产生 1 的贡献。
- 若在这条边之前的点集 S ，那么这条边存在的概率为：

$$\left(\frac{1}{2}\right)^{|S|(n-|S|)} \times \prod_{u_i \in S, v_i \notin S} 2p_i \quad (5)$$

- 直接枚举点集 S ，时间复杂度 $\mathcal{O}(2^n)$ 。

- 由于竞赛图缩点之后是链，答案为这条链的期望点数。考虑枚举链上的每一条边，每条边都会对答案产生 1 的贡献。
- 若在这条边之前的点集 S ，那么这条边存在的概率为：

$$\left(\frac{1}{2}\right)^{|S|(n-|S|)} \times \prod_{u_i \in S, v_i \notin S} 2p_i \quad (5)$$

- 直接枚举点集 S ，时间复杂度 $\mathcal{O}(2^n)$ 。
- 但是可以只对每个弱连通分量求解，最后将答案暴力卷积即可。时间复杂度 $\mathcal{O}(2^m)$ 。

- 对每个 $i = 1, 2, \dots, n$, 求点数为 i 的随机强连通竞赛图的哈密顿回路数量期望, 对 998244353 取模。
- $n \leq 10^5$ 。

- 考虑所有 n 个点的竞赛图的哈密顿回路总数为 $(n-1)! \times 2^{\binom{n}{2}-n}$ 。

- 考虑所有 n 个点的竞赛图的哈密顿回路总数为 $(n-1)! \times 2^{\binom{n}{2}-n}$ 。
- 哈密顿回路数量相当于圆排列个数，每个哈密顿回路固定 n 条边，其余边方向任意。

- 考虑所有 n 个点的竞赛图的哈密顿回路总数为 $(n-1)! \times 2^{\binom{n}{2}-n}$ 。
- 哈密顿回路数量相当于圆排列个数，每个哈密顿回路固定 n 条边，其余边方向任意。
- 现在问题相当于求强连通竞赛图的个数 F_n ，有：

$$F_n = 2^{\binom{n}{2}} - \sum_{i=1}^{n-1} F_i \times 2^{\binom{n-i}{2}} \times \binom{n}{i} \quad (6)$$

- 考虑所有 n 个点的竞赛图的哈密顿回路总数为 $(n-1)! \times 2^{\binom{n}{2}-n}$ 。
- 哈密顿回路数量相当于圆排列个数，每个哈密顿回路固定 n 条边，其余边方向任意。
- 现在问题相当于求强连通竞赛图的个数 F_n ，有：

$$F_n = 2^{\binom{n}{2}} - \sum_{i=1}^{n-1} F_i \times 2^{\binom{n-i}{2}} \times \binom{n}{i} \quad (6)$$

- 使用分治 FFT 或 多项式求逆即可，时间复杂度 $\mathcal{O}(n \log^2 n)$ 或 $\mathcal{O}(n \log n)$ 。

- 定义一个竞赛图的比分序列，是把竞赛图每个点的出度从小到大排列得到的序列。一个长度为 n 的序列 $d = (d_1, d_2, \dots, d_n)$ ，是合法的比分序列当且仅当：

$$\forall i \in [1, n], s_i \geq \frac{i(i-1)}{2} \quad (7)$$

- 定义一个竞赛图的比分序列，是把竞赛图每个点的出度从小到大排列得到的序列。一个长度为 n 的序列 $d = (d_1, d_2, \dots, d_n)$ ，是合法的比分序列当且仅当：

$$\forall i \in [1, n], s_i \geq \frac{i(i-1)}{2} \quad (7)$$

- 其中 s 为 d 的前缀和数组。

- 定义一个竞赛图的比分序列，是把竞赛图每个点的出度从小到大排列得到的序列。一个长度为 n 的序列 $d = (d_1, d_2, \dots, d_n)$ ，是合法的比分序列当且仅当：

$$\forall i \in [1, n], s_i \geq \frac{i(i-1)}{2} \quad (7)$$

- 其中 s 为 d 的前缀和数组。
- 另外，所有 $s_i = \frac{i(i-1)}{2}$ 的位置为 SCC 的分隔点。

- 必要性：考虑这 i 个点的导出子图， s_i 至少包含了这些边。

- 必要性：考虑这 i 个点的导出子图， s_i 至少包含了这些边。
- 充分性：先构造一张竞赛图，对于所有 $i > j$ 的 (i, j) 连边，现在其比分序列满足 $u_i = i - 1$ 。

- 必要性：考虑这 i 个点的导出子图， s_i 至少包含了这些边。
- 充分性：先构造一张竞赛图，对于所有 $i > j$ 的 (i, j) 连边，现在其比分序列满足 $u_i = i - 1$ 。
- 每次找到第一个 u 与 d 不同的位置 i 进行调整，这个位置一定满足 $u_i < d_i$ 。

- 必要性：考虑这 i 个点的导出子图， s_i 至少包含了这些边。
- 充分性：先构造一张竞赛图，对于所有 $i > j$ 的 (i, j) 连边，现在其比分序列满足 $u_i = i - 1$ 。
- 每次找到第一个 u 与 d 不同的位置 i 进行调整，这个位置一定满足 $u_i < d_i$ 。
- 找到最大的 j 满足 $u_j = u_i$ ，再找到第一个 j 之后的 k 满足 $u_k > d_k$ 。由于 $i \leq j < k$ ， $u_i < d_i \leq d_k < u_k$ ，所以 $u_k - u_i \geq 2$ ，一定存在一个位置 p 满足边 $(k, p), (p, i)$ 存在，翻转这两条边即可。

- 必要性：考虑这 i 个点的导出子图， s_i 至少包含了这些边。
- 充分性：先构造一张竞赛图，对于所有 $i > j$ 的 (i, j) 连边，现在其比分序列满足 $u_i = i - 1$ 。
- 每次找到第一个 u 与 d 不同的位置 i 进行调整，这个位置一定满足 $u_i < d_i$ 。
- 找到最大的 j 满足 $u_j = u_i$ ，再找到第一个 j 之后的 k 满足 $u_k > d_k$ 。由于 $i \leq j < k$ ， $u_i < d_i \leq d_k < u_k$ ，所以 $u_k - u_i \geq 2$ ，一定存在一个位置 p 满足边 $(k, p), (p, i)$ 存在，翻转这两条边即可。
- 可以发现这个过程中除了当前 u_i 相同的连续段是逐渐抬升的，其余 u 的位置都仍满足单调。

- 定义一个竞赛图的度数集合是由该竞赛图中每个点的出度所构成的集合。
- 给定一个大小为 m 的集合 S ，判断是否存在一个竞赛图的度数集合为 S 。如果存在，构造一个点数最小的满足条件的竞赛图。
- $m \leq 31, S_i \leq 30$ 。

- 由于 $\sum_{i=1}^n d_i = \frac{n(n-1)}{2}$, $d_j \leq 30$, 所以 $n(n-1) \leq 2 \times 30n$, 得到 $n \leq 61$ 。

- 由于 $\sum_{i=1}^n d_i = \frac{n(n-1)}{2}$, $d_j \leq 30$, 所以 $n(n-1) \leq 2 \times 30n$, 得到 $n \leq 61$ 。
- 背包出一组合法比分序列, 使用兰道定理构造解即可。

- 由于 $\sum_{i=1}^n d_i = \frac{n(n-1)}{2}$, $d_j \leq 30$, 所以 $n(n-1) \leq 2 \times 30n$, 得到 $n \leq 61$ 。
- 背包出一组合法比分序列, 使用兰道定理构造解即可。
- 时间复杂度 $\mathcal{O}(n^4)$, 构造部分的复杂度是 $\mathcal{O}(n^3)$ 的。

- 由于 $\sum_{i=1}^n d_i = \frac{n(n-1)}{2}$, $d_j \leq 30$, 所以 $n(n-1) \leq 2 \times 30n$, 得到 $n \leq 61$ 。
- 背包出一组合法比分序列, 使用兰道定理构造解即可。
- 时间复杂度 $\mathcal{O}(n^4)$, 构造部分的复杂度是 $\mathcal{O}(n^3)$ 的。
- 但是有一件比较神奇的事情, 调整时可以直接找到 i, j, k 满足 $u_i > d_i, u_j = d_j, u_k < d_k$, 存在边 $(k, j), (j, i)$ 则反转。不断重复此过程直到无需调整。

- 由于 $\sum_{i=1}^n d_i = \frac{n(n-1)}{2}$, $d_j \leq 30$, 所以 $n(n-1) \leq 2 \times 30n$, 得到 $n \leq 61$ 。
- 背包出一组合法比分序列, 使用兰道定理构造解即可。
- 时间复杂度 $\mathcal{O}(n^4)$, 构造部分的复杂度是 $\mathcal{O}(n^3)$ 的。
- 但是有一件比较神奇的事情, 调整时可以直接找到 i, j, k 满足 $u_i > d_i, u_j = d_j, u_k < d_k$, 存在边 $(k, j), (j, i)$ 则反转。不断重复此过程直到无需调整。
- 时间复杂度未知, 正确性未知, 至少它过了, 我目前还无法证明或证伪正确性。

- 由于 $\sum_{i=1}^n d_i = \frac{n(n-1)}{2}$, $d_j \leq 30$, 所以 $n(n-1) \leq 2 \times 30n$, 得到 $n \leq 61$ 。
- 背包出一组合法比分序列, 使用兰道定理构造解即可。
- 时间复杂度 $\mathcal{O}(n^4)$, 构造部分的复杂度是 $\mathcal{O}(n^3)$ 的。
- 但是有一件比较神奇的事情, 调整时可以直接找到 i, j, k 满足 $u_i > d_i, u_j = d_j, u_k < d_k$, 存在边 $(k, j), (j, i)$ 则反转。不断重复此过程直到无需调整。
- 时间复杂度未知, 正确性未知, 至少它过了, 我目前还无法证明或证伪正确性。
- 有好鸽鸽懂的话可以指导我一下!

- 给定一张竞赛图，定义一次操作为选取一个顶点并翻转其所有邻边的方向。
- 求出是否存在一个操作序列使得这个图强连通。若存在求出最小操作次数，以及操作次数最小的操作方案数，对 998244353 取模。
- $n \leq 2 \times 10^3$ 。

- 引理: $n \geq 4$ 阶强连通竞赛图存在 $n - 1$ 阶强连通子图。

- 引理: $n \geq 4$ 阶强连通竞赛图存在 $n - 1$ 阶强连通子图。
- 证明: 考虑 n 阶强连通竞赛图 G , 其去掉节点 n 的结果是 $n - 1$ 阶竞赛图 G' , 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。

- 引理: $n \geq 4$ 阶强连通竞赛图存在 $n - 1$ 阶强连通子图。
- 证明: 考虑 n 阶强连通竞赛图 G , 其去掉节点 n 的结果是 $n - 1$ 阶竞赛图 G' , 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$, 则 G' 即为 $n - 1$ 阶强连通子图。

- 引理: $n \geq 4$ 阶强连通竞赛图存在 $n - 1$ 阶强连通子图。
- 证明: 考虑 n 阶强连通竞赛图 G , 其去掉节点 n 的结果是 $n - 1$ 阶竞赛图 G' , 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$, 则 G' 即为 $n - 1$ 阶强连通子图。
- 否则必然存在 $u \in S_k, v \in S_1$, 且存在边 $(u, n), (n, v)$ 。

- 引理: $n \geq 4$ 阶强连通竞赛图存在 $n - 1$ 阶强连通子图。
- 证明: 考虑 n 阶强连通竞赛图 G , 其去掉节点 n 的结果是 $n - 1$ 阶竞赛图 G' , 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$, 则 G' 即为 $n - 1$ 阶强连通子图。
- 否则必然存在 $u \in S_k, v \in S_1$, 且存在边 $(u, n), (n, v)$ 。
- 由于 $n \geq 4$, 我们可以找到一个点 $w \notin \{u, v, n\}$, 满足原图中任意两点 $a, b \neq w$ 都有不经过 w 的路径 $a \rightarrow u \rightarrow n \rightarrow v \rightarrow b$, 即 G 去掉 w 后为 $n - 1$ 阶强连通子图。

- 引理: $n \geq 4$ 阶强连通竞赛图存在 $n - 1$ 阶强连通子图。
- 证明: 考虑 n 阶强连通竞赛图 G , 其去掉节点 n 的结果是 $n - 1$ 阶竞赛图 G' , 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$, 则 G' 即为 $n - 1$ 阶强连通子图。
- 否则必然存在 $u \in S_k, v \in S_1$, 且存在边 $(u, n), (n, v)$ 。
- 由于 $n \geq 4$, 我们可以找到一个点 $w \notin \{u, v, n\}$, 满足原图中任意两点 $a, b \neq w$ 都有不经过 w 的路径 $a \rightarrow u \rightarrow n \rightarrow v \rightarrow b$, 即 G 去掉 w 后为 $n - 1$ 阶强连通子图。
- 推论: $n \geq 4$ 阶强连通竞赛图总可以翻转恰好一个点, 使得它还是强连通的。

- 结论: $n \geq 7$ 阶竞赛图总可以通过翻转至多一个点变为强连通。

- 结论: $n \geq 7$ 阶竞赛图总可以通过翻转至多一个点变为强连通。
- 证明: 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。

CF1268D Invertation in Tournament

- 结论: $n \geq 7$ 阶竞赛图总可以通过翻转至多一个点变为强连通。
- 证明: 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$ 则无需翻转。

- 结论: $n \geq 7$ 阶竞赛图总可以通过翻转至多一个点变为强连通。
- 证明: 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$ 则无需翻转。
- 若 $k \geq 3$ 则任选 $w \in S_m (1 < m < k)$ 翻转, 任意两点 $a, b \neq w$ 间存在路径 $a \rightarrow u \rightarrow w \rightarrow v \rightarrow b$ 其中 $u \in S_k, v \in S_1$ 。

- 结论: $n \geq 7$ 阶竞赛图总可以通过翻转至多一个点变为强连通。
- 证明: 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$ 则无需翻转。
- 若 $k \geq 3$ 则任选 $w \in S_m (1 < m < k)$ 翻转, 任意两点 $a, b \neq w$ 间存在路径 $a \rightarrow u \rightarrow w \rightarrow v \rightarrow b$ 其中 $u \in S_k, v \in S_1$ 。
- 若 $k = 2$, 由于 $n \geq 7$, S_1, S_2 中必存在一个点数大于等于 4, 假设其为 S_1 (S_2 同理)。

- 结论: $n \geq 7$ 阶竞赛图总可以通过翻转至多一个点变为强连通。
- 证明: 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$ 则无需翻转。
- 若 $k \geq 3$ 则任选 $w \in S_m (1 < m < k)$ 翻转, 任意两点 $a, b \neq w$ 间存在路径 $a \rightarrow u \rightarrow w \rightarrow v \rightarrow b$ 其中 $u \in S_k, v \in S_1$ 。
- 若 $k = 2$, 由于 $n \geq 7$, S_1, S_2 中必存在一个点数大于等于 4, 假设其为 S_1 (S_2 同理)。
- 根据引理, 我们可以找到一个 S_1 中的点, 使得其翻转后 S_1 强连通, 然后 S_1, S_2 相互之间都有连向的边, 得到一个强连通图。

- 结论: $n \geq 7$ 阶竞赛图总可以通过翻转至多一个点变为强连通。
- 证明: 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$ 则无需翻转。
- 若 $k \geq 3$ 则任选 $w \in S_m (1 < m < k)$ 翻转, 任意两点 $a, b \neq w$ 间存在路径 $a \rightarrow u \rightarrow w \rightarrow v \rightarrow b$ 其中 $u \in S_k, v \in S_1$ 。
- 若 $k = 2$, 由于 $n \geq 7$, S_1, S_2 中必存在一个点数大于等于 4, 假设其为 S_1 (S_2 同理)。
- 根据引理, 我们可以找到一个 S_1 中的点, 使得其翻转后 S_1 强连通, 然后 S_1, S_2 相互之间都有连向的边, 得到一个强连通图。
- 对于 $n < 7$, 直接暴力。对于 $n \geq 7$, 先判断强连通, 然后枚举翻转的点, 通过比分序列求出是否有多于一个 SCC 即可。

CF1268D Invertation in Tournament

- 结论: $n \geq 7$ 阶竞赛图总可以通过翻转至多一个点变为强连通。
- 证明: 设对其缩点后得到的 SCC 序列为 $S_1, S_2, \dots, S_k$ 。
- 若 $k = 1$ 则无需翻转。
- 若 $k \geq 3$ 则任选 $w \in S_m (1 < m < k)$ 翻转, 任意两点 $a, b \neq w$ 间存在路径 $a \rightarrow u \rightarrow w \rightarrow v \rightarrow b$ 其中 $u \in S_k, v \in S_1$ 。
- 若 $k = 2$, 由于 $n \geq 7$, S_1, S_2 中必存在一个点数大于等于 4, 假设其为 S_1 (S_2 同理)。
- 根据引理, 我们可以找到一个 S_1 中的点, 使得其翻转后 S_1 强连通, 然后 S_1, S_2 相互之间都有连向的边, 得到一个强连通图。
- 对于 $n < 7$, 直接暴力。对于 $n \geq 7$, 先判断强连通, 然后枚举翻转的点, 通过比分序列求出是否有多于一个 SCC 即可。
- 需要对比分序列排序, 时间复杂度 $\mathcal{O}(n^2 \log n)$ 。也可以使用归并来做到 $\mathcal{O}(n^2)$ 。

弦图

定义

- 子图：点集和边集均为原图点集和边集子集的图。

定义

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。

定义

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。
- 团：完全子图。

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。
- 团：完全子图。
- 极大团：不是其他团子图的团。

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。
- 团：完全子图。
- 极大团：不是其他团子图的团。
- 最大团：点数最大的团。

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。
- 团：完全子图。
- 极大团：不是其他团子图的团。
- 最大团：点数最大的团。
- 团数：最大团的点数。

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。
- 团：完全子图。
- 极大团：不是其他团子图的团。
- 最大团：点数最大的团。
- 团数：最大团的点数。
- 最小染色：用最少的颜色给点染色使得所有边连接的两点颜色不同。

定义

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。
- 团：完全子图。
- 极大团：不是其他团子图的团。
- 最大团：点数最大的团。
- 团数：最大团的点数。
- 最小染色：用最少的颜色给点染色使得所有边连接的两点颜色不同。
- 色数：最小染色的颜色数。

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。
- 团：完全子图。
- 极大团：不是其他团子图的团。
- 最大团：点数最大的团。
- 团数：最大团的点数。
- 最小染色：用最少的颜色给点染色使得所有边连接的两点颜色不同。
- 色数：最小染色的颜色数。
- 最大独立集：最大的点集使得点集中任意两点都没有边直接相连。

- 子图：点集和边集均为原图点集和边集子集的图。
- 导出子图（诱导子图）：点集为原图点集子集，边集为所有满足两个端点均在选定点集中的边构成集合的图。
- 团：完全子图。
- 极大团：不是其他团子图的团。
- 最大团：点数最大的团。
- 团数：最大团的点数。
- 最小染色：用最少的颜色给点染色使得所有边连接的两点颜色不同。
- 色数：最小染色的颜色数。
- 最大独立集：最大的点集使得点集中任意两点都没有边直接相连。
- 最小团覆盖：用最少的团覆盖所有的点。

- 弦：连接环中不相邻两点的边。

弦图的定义与性质

- 弦：连接环中不相邻两点的边。
- 弦图：任意长度大于 3 的环都有一个弦的图称为弦图。

弦图的定义与性质

- 弦：连接环中不相邻两点的边。
- 弦图：任意长度大于 3 的环都有一个弦的图称为弦图。
- 团数 $\leq$ 色数：对最大团进行染色需要团数种颜色。

弦图的定义与性质

- 弦：连接环中不相邻两点的边。
- 弦图：任意长度大于 3 的环都有一个弦的图称为弦图。
- 团数 $\leq$ 色数：对最大团进行染色需要团数种颜色。
- 最大独立集 $\leq$ 最小团覆盖：每个团中至多选择一个点。

弦图的定义与性质

- 弦：连接环中不相邻两点的边。
- 弦图：任意长度大于 3 的环都有一个弦的图称为弦图。
- 团数 $\leq$ 色数：对最大团进行染色需要团数种颜色。
- 最大独立集 $\leq$ 最小团覆盖：每个团中至多选择一个点。
- 弦图的任意导出子图一定是弦图：导出子图上存在的无弦环，原图上也存在。

引理 1

弦图上任意两点 x, y 间的极小点割集（删去后 x, y 不连通且不存在点割集子集）的导出子图一定是一个团。

引理 1

弦图上任意两点 x, y 间的极小点割集（删去后 x, y 不连通且不存在点割集子集）的导出子图一定是一个团。

证明.

考虑删去点割集后的图，每个点割集内的点一定和 x, y 所在连通块均有边，否则可以从点割集中删去这个点。

对于极小点割集中任意两点 a, b ，它们分别有邻居 a_1, a_2 和 b_1, b_2 满足 a_1, b_1 与 x 连通， a_2, b_2 与 y 连通。

考虑环 $a - a_1 \sim b_1 - b - b_2 \sim a_2 - a$ ，其中 $\sim$ 是最短路。

这个环一定存在一条弦，只能是 (a, b) 。

由此得到极小点割集中任意两点均有连边。



单纯点

- 定义：相邻点加上本身的导出子图是团的点。

单纯点

- 定义：相邻点加上本身的导出子图是团的点。

引理 2

任何一个弦图至少一个不相邻的单纯点，非完全图的弦图至少两个。

单纯点

- 定义：相邻点加上本身的导出子图是团的点。

引理 2

任何一个弦图至少一个不相邻的单纯点，非完全图的弦图至少两个。

证明.

考虑归纳，当点数小于等于 3 时，结论显然成立。

当点数大于 3 时，对于完全图，它的每个点都是单纯点。

对于非完全图的弦图，找到其一对不相邻点 u, v ，设除去极小点割集 I 后 u, v 所在的连通块为 A, B 。对于原图的导出子图 $A + I, B + I$ 若有至少一个是完全图，则对应的 u 或 v 是单纯点，否则可以在 $A + I$ 中找到两个不相邻的单纯点。

因为 I 是完全图，所以 A 中的单纯点在原图上也是单纯点。

每次会使得问题规模缩小 ($A + I$ 不包含 v)。 □

- 定义：一个点的排列 $p_1, p_2, \dots, p_n$ 满足 p_i 在 $p_i, p_{i+1}, \dots, p_n$ 的导出子图中为一个单纯点。

- 定义：一个点的排列 $p_1, p_2, \dots, p_n$ 满足 p_i 在 $p_i, p_{i+1}, \dots, p_n$ 的导出子图中为一个单纯点。
- 弦图一定存在完美消除序列：每次取单纯点即可构造。

- 定义：一个点的排列 $p_1, p_2, \dots, p_n$ 满足 p_i 在 $p_i, p_{i+1}, \dots, p_n$ 的导出子图中为一个单纯点。
- 弦图一定存在完美消除序列：每次取单纯点即可构造。
- 存在完美消除序列的图一定是弦图：每个环在序列中第一次出现的点一定与环上相邻点有边。根据完美消除序列的定义，这两个相邻点一定有边。

- 定义：一个点的排列 $p_1, p_2, \dots, p_n$ 满足 p_i 在 $p_i, p_{i+1}, \dots, p_n$ 的导出子图中为一个单纯点。
- 弦图一定存在完美消除序列：每次取单纯点即可构造。
- 存在完美消除序列的图一定是弦图：每个环在序列中第一次出现的点一定与环上相邻点有边。根据完美消除序列的定义，这两个相邻点一定有边。
- 由此我们得到一个无向图是弦图 $\Leftrightarrow$ 其存在完全消除序列。

- 直接暴力求解完美消除序列是 $\mathcal{O}(n^4)$ 的。

- 直接暴力求解完美消除序列是 $\mathcal{O}(n^4)$ 的。
- MCS 最大势算法可以在 $\mathcal{O}(n + m)$ 的时间复杂度内求出弦图的完美消除序列：

- 直接暴力求解完美消除序列是 $\mathcal{O}(n^4)$ 的。
- MCS 最大势算法可以在 $\mathcal{O}(n + m)$ 的时间复杂度内求出弦图的完美消除序列：
 1. 每次找到一个未标号点 i ，使得其相连的已标号点数量 l_i 最多。

- 直接暴力求解完美消除序列是 $\mathcal{O}(n^4)$ 的。
- MCS 最大势算法可以在 $\mathcal{O}(n + m)$ 的时间复杂度内求出弦图的完美消除序列：
 1. 每次找到一个未标号点 i ，使得其相连的已标号点数量 l_i 最多。
 2. 对 i 标号，然后更新所有相连点的 l 。

- 直接暴力求解完美消除序列是 $\mathcal{O}(n^4)$ 的。
- MCS 最大势算法可以在 $\mathcal{O}(n + m)$ 的时间复杂度内求出弦图的完美消除序列：
 1. 每次找到一个未标号点 i ，使得其相连的已标号点数量 l_i 最多。
 2. 对 i 标号，然后更新所有相连点的 l 。
 3. 用链表或 vector 对每个 x 维护 $l_i = x$ 的 i 。

- 直接暴力求解完美消除序列是 $\mathcal{O}(n^4)$ 的。
- MCS 最大势算法可以在 $\mathcal{O}(n + m)$ 的时间复杂度内求出弦图的完美消除序列：
 1. 每次找到一个未标号点 i ，使得其相连的已标号点数量 l_i 最多。
 2. 对 i 标号，然后更新所有相连点的 l 。
 3. 用链表或 vector 对每个 x 维护 $l_i = x$ 的 i 。
 4. 把最后求出的序列倒过来。

- 直接暴力求解完美消除序列是 $\mathcal{O}(n^4)$ 的。
- MCS 最大势算法可以在 $\mathcal{O}(n + m)$ 的时间复杂度内求出弦图的完美消除序列：
 1. 每次找到一个未标号点 i ，使得其相连的已标号点数量 l_i 最多。
 2. 对 i 标号，然后更新所有相连点的 l 。
 3. 用链表或 vector 对每个 x 维护 $l_i = x$ 的 i 。
 4. 把最后求出的序列倒过来。
- 判断求出的序列是否合法：对于每个 i ，记在 $p_i, p_{i+1}, \dots, p_n$ 中与 p_i 相连的点依次为 $p_{c_1}, p_{c_2}, \dots, p_{c_k}$ ，判断 p_{c_1} 是否与 $p_{c_i} (i > 1)$ 相连。

- 直接暴力求解完美消除序列是 $\mathcal{O}(n^4)$ 的。
- MCS 最大势算法可以在 $\mathcal{O}(n + m)$ 的时间复杂度内求出弦图的完美消除序列：
 1. 每次找到一个未标号点 i ，使得其相连的已标号点数量 l_i 最多。
 2. 对 i 标号，然后更新所有相连点的 l 。
 3. 用链表或 vector 对每个 x 维护 $l_i = x$ 的 i 。
 4. 把最后求出的序列倒过来。
- 判断求出的序列是否合法：对于每个 i ，记在 $p_i, p_{i+1}, \dots, p_n$ 中与 p_i 相连的点依次为 $p_{c_1}, p_{c_2}, \dots, p_{c_k}$ ，判断 p_{c_1} 是否与 $p_{c_i} (i > 1)$ 相连。
- 证明：团中其余的边会在 $p_{c_2}, p_{c_3}, \dots, p_{c_k}$ 处依次被判断。

- 设 $w(x)$ 为 x 在 MCS 最大势算法结果序列中的位置。我们需要证明对于任何一个弦图，算法求出的序列一定是一个完美消除序列，即在序列中位于某个点后面且与这个点相连的所有点两两相连。

- 设 $w(x)$ 为 x 在 MCS 最大势算法结果序列中的位置。我们需要证明对于任何一个弦图，算法求出的序列一定是一个完美消除序列，即在序列中位于某个点后面且与这个点相连的所有点两两相连。

引理 3

考虑三个点 x, y, z 满足 $w(x) < w(y) < w(z)$ 且 y, z 不相连， x, z 相连。由于 y 先于 x 加入，一定存在一个点 p 使得 $w(y) < w(p)$ 且 x, p 不相连， y, p 相连。

引理 4

任意一个弦图一定不存在一个序列 $v_0, v_1, \dots, v_k (k \geq 2)$ 满足下列性质:

1. v_i, v_j 相连当且仅当 $|i - j| = 1$ 。
2. $w(v_0) > w(v_i), i \in [1, k]$ 。
3. 存在 $i \in [1, k - 1]$, 满足 $w(v_i) < w(v_{i+1}) < \dots < w(v_k)$ 且 $w(v_i) < w(v_{i-1}) < \dots < w(v_1) < w(v_k)$ 。

MCS 最大势算法正确性证明

证明.

由于 $w(v_1) < w(v_k) < w(v_0)$, 且 v_1, v_0 相连, v_k, v_0 不相连, 由引理 3 得到存在 x 满足 $w(v_k) < w(x)$ 且 v_k, x 相连, v_1, x 不相连。

考虑最小的 $j \in (1, k]$ 满足 v_j, x 相连, 我们可以推出 v_0, x 不相连, 否则 $v_0, v_1 \cdots v_j, x$ 构成了一个长度 ≥ 4 且无弦的环。

若 $x < v_0$, 则 $v_0, v_1, \dots, v_j, x$ 也是一个满足性质的序列;

若 $v_0 < x$, 则 $x, v_j, \dots, v_1, v_0$ 也是一个满足性质的序列。

在上面的推导中, 我们扩大了 $\min(v_0, v_k)$, 于是一直推下去, 一定会产生矛盾。 □

MCS 最大势算法正确性证明

证明.

由于 $w(v_1) < w(v_k) < w(v_0)$, 且 v_1, v_0 相连, v_k, v_0 不相连, 由引理 3 得到存在 x 满足 $w(v_k) < w(x)$ 且 v_k, x 相连, v_1, x 不相连。

考虑最小的 $j \in (1, k]$ 满足 v_j, x 相连, 我们可以推出 v_0, x 不相连, 否则 $v_0, v_1 \cdots v_j, x$ 构成了一个长度 ≥ 4 且无弦的环。

若 $x < v_0$, 则 $v_0, v_1, \dots, v_j, x$ 也是一个满足性质的序列;

若 $v_0 < x$, 则 $x, v_j, \dots, v_1, v_0$ 也是一个满足性质的序列。

在上面的推导中, 我们扩大了 $\min(v_0, v_k)$, 于是一直推下去, 一定会产生矛盾。 □

- 弦图让我感受到了万物皆可归纳-----

定理 5

对于任何一个弦图，最大势算法求出的序列一定是一个完美消除序列。

定理 5

对于任何一个弦图，最大势算法求出的序列一定是一个完美消除序列。

证明.

考虑任意三个点 x, y, z 满足 $w(x) < w(y) < w(z)$ ，我们需要证明若 x, y 相连， x, z 相连，则 y, z 一定相连。

考虑反证法，假设不相连，那么 z, x, y 就是一个满足引理 4 中性质的序列，我们证明了这样序列不存在，所以矛盾， y, z 相连。 □

定理 5

对于任何一个弦图，最大势算法求出的序列一定是一个完美消除序列。

证明.

考虑任意三个点 x, y, z 满足 $w(x) < w(y) < w(z)$ ，我们需要证明若 x, y 相连， x, z 相连，则 y, z 一定相连。

考虑反证法，假设不相连，那么 z, x, y 就是一个满足引理 4 中性质的序列，我们证明了这样序列不存在，所以矛盾， y, z 相连。 □

- 板子：SP5446 FISHNET - Fishing Net。

- 令 $N(x)$ 为满足与 x 直接有边相连且在完美消除序列上的 x 之后的序列。则弦图的极大团一定能表示为 $\{x\} + N(x)$ 。

- 令 $N(x)$ 为满足与 x 直接有边相连且在完美消除序列上的 x 之后的序列。则弦图的极大团一定能表示为 $\{x\} + N(x)$ 。
- 由此可得弦图的团数即为每个点 $|x + N(x)|$ 的最大值。

- 令 $N(x)$ 为满足与 x 直接有边相连且在完美消除序列上的 x 之后的序列。则弦图的极大团一定能表示为 $\{x\} + N(x)$ 。
- 由此可得弦图的团数即为每个点 $|x + N(x)|$ 的最大值。
- 在弦图中色数与团数相同，考虑按完美消除序列的倒序贪心染色 mex 即可。

- 令 $N(x)$ 为满足与 x 直接有边相连且在完美消除序列上的 x 之后的序列。则弦图的极大团一定能表示为 $\{x\} + N(x)$ 。
- 由此可得弦图的团数即为每个点 $|x + N(x)|$ 的最大值。
- 在弦图中色数与团数相同，考虑按完美消除序列的倒序贪心染色 mex 即可。
- 证明：团数 $\leq$ 色数，由此方法构造能取到等号。

- 令 $N(x)$ 为满足与 x 直接有边相连且在完美消除序列上的 x 之后的序列。则弦图的极大团一定能表示为 $\{x\} + N(x)$ 。
- 由此可得弦图的团数即为每个点 $|x + N(x)|$ 的最大值。
- 在弦图中色数与团数相同，考虑按完美消除序列的倒序贪心染色 mex 即可。
- 证明：团数 $\leq$ 色数，由此方法构造能取到等号。
- 弦图色数板子：HNOI2008 神奇的国度。

- 最大独立集：完美消除序列从前往后，贪心选择所有没有与已经选择的点有直接连边的点。

- 最大独立集：完美消除序列从前往后，贪心选择所有没有与已经选择的点有直接连边的点。
- 证明：考虑当前选中最靠前的极大团，被选中的点只与团中的点有边，选团中其它点一定不优于选这个点，并且整个团都不选点显然不优。

最大独立集与最小团覆盖

- 最大独立集：完美消除序列从前往后，贪心选择所有没有与已经选择的点有直接连边的点。
- 证明：考虑当前选中最靠前的极大团，被选中的点只与团中的点有边，选团中其它点一定不优于选这个点，并且整个团都不选点显然不优。
- 最小团覆盖：最大独立集中每个点 v 的 $v + N(v)$ 所组成的集合。

最大独立集与最小团覆盖

- 最大独立集：完美消除序列从前往后，贪心选择所有没有与已经选择的点有直接连边的点。
- 证明：考虑当前选中最靠前的极大团，被选中的点只与团中的点有边，选团中其它点一定不优于选这个点，并且整个团都不选点显然不优。
- 最小团覆盖：最大独立集中每个点 v 的 $v + N(v)$ 所组成的集合。
- 证明：最大独立集 $\leq$ 最小团覆盖，由此方法构造能取到等号。

最大独立集与最小团覆盖

- 最大独立集：完美消除序列从前往后，贪心选择所有没有与已经选择的点有直接连边的点。
- 证明：考虑当前选中最靠前的极大团，被选中的点只与团中的点有边，选团中其它点一定不优于选这个点，并且整个团都不选点显然不优。
- 最小团覆盖：最大独立集中每个点 v 的 $v + N(v)$ 所组成的集合。
- 证明：最大独立集 $\leq$ 最小团覆盖，由此方法构造能取到等号。
- 弦图最大独立集板子：TJOI2007 小朋友

- 将区间视为点，有交的区间之间有边的图称为区间图。

- 将区间视为点，有交的区间之间有边的图称为区间图。
- 区间图一定是弦图，它的一个完美消除序列为按右端点从小到大排序的结果。

- 将区间视为点，有交的区间之间有边的图称为区间图。
- 区间图一定是弦图，它的一个完美消除序列为按右端点从小到大排序的结果。
- 用这个东西可以证明很多贪心的正确性，但是我的评价是大炮打蚊子。

平面图

定义 9.1.1 (平面图)

在一个平面上，如果能够画出无向图 G 的图解，其中没有任何边的交叉，则称图 G 是个平面图。

定义 9.1.1 (平面图)

在一个平面上，如果能够画出无向图 G 的图解，其中没有任何边的交叉，则称图 G 是个平面图。

- $K_{3,3}$ 和 K_5 分别是边最少的非平面图和点最少的非平面图。

定义 9.1.1 (平面图)

在一个平面上，如果能够画出无向图 G 的图解，其中没有任何边的交叉，则称图 G 是个平面图。

- $K_{3,3}$ 和 K_5 分别是边最少的非平面图和点最少的非平面图。
- 设 G 是平面图，由 G 的边将 G 所在的平面划分成若干个区域，每个区域称为 G 的一个面，其中面积无限的面称为无限面或外部面，面积有限的称为有限面或内部面。包围每个面的所有边组成的回路称为该面的边界，边界的长度称为该面的次数。

- 对于任意连通平面图 G 有: $n - m + r = 2$ 。其中 n, m, r 分别为 G 的点数、边数和面数。

- 对于任意连通平面图 G 有: $n - m + r = 2$ 。其中 n, m, r 分别为 G 的点数、边数和面数。
- 类似我们可以得到不连通时的情况, 若连通块数为 p 则有:
$$n - m + r = p + 1。$$

- 对于任意连通平面图 G 有: $n - m + r = 2$ 。其中 n, m, r 分别为 G 的点数、边数和面数。
- 类似我们可以得到不连通时的情况, 若连通块数为 p 则有:
$$n - m + r = p + 1。$$
- 若一张平面图中所有面的次数均不小于 l , 则有推论:
$$m \leq \frac{l}{l-2}(n - p - 1)$$

欧拉公式

- 对于任意连通平面图 G 有: $n - m + r = 2$ 。其中 n, m, r 分别为 G 的点数、边数和面数。
- 类似我们可以得到不连通时的情况, 若连通块数为 p 则有:
$$n - m + r = p + 1。$$
- 若一张平面图中所有面的次数均不小于 l , 则有推论:
$$m \leq \frac{l}{l-2}(n - p - 1)$$
- 我们还可以利用这个公式推出一张 n 个点的平面图的边数不超过 $3n - 6$ 。

定理 9.3.1 (库拉托夫斯基定理)

图 G 是平面图当且仅当图中不存在任何与 $K_{3,3}$ 和 K_5 同胚的子图。

定理 9.3.1 (库拉托夫斯基定理)

图 G 是平面图当且仅当图中不存在任何与 $K_{3,3}$ 和 K_5 同胚的子图。

- 这样我们就有了一个在多项式时间复杂度内判断一张图是否是平面图的算法。

定义 9.4.1 (对偶图)

- 对于平面图 G , 对偶图 G^* 有以下特性:

定义 9.4.1 (对偶图)

- 对于平面图 G , 对偶图 G^* 有以下特性:
 - G^* 的每个顶点对应 G 中的一个面。

定义 9.4.1 (对偶图)

- 对于平面图 G , 对偶图 G^* 有以下特性:
 - G^* 的每个顶点对应 G 中的一个面。
 - G^* 的每条边对应 G 中的两个相邻的面之间的边。

定义 9.4.1 (对偶图)

- 对于平面图 G ，对偶图 G^* 有以下特性：
 - G^* 的每个顶点对应 G 中的一个面。
 - G^* 的每条边对应 G 中的两个相邻的面之间的边。
 - 如果 G 中的两个面共享一条边，那么 G^* 中对应的两个顶点之间有一条边。

定义 9.4.1 (对偶图)

- 对于平面图 G , 对偶图 G^* 有以下特性:
 - G^* 的每个顶点对应 G 中的一个面。
 - G^* 的每条边对应 G 中的两个相邻的面之间的边。
 - 如果 G 中的两个面共享一条边, 那么 G^* 中对应的两个顶点之间有一条边。
- 显然对偶图也是平面图, 而且原图的点、边、面分别对应偶图的面、边、点。

- 平面图转对偶图的方法。

- 平面图转对偶图的方法。
- 大致想法是把一条无向边拆成两条有向边，然后确定每条有向边所在的面，并把一条边所在的面和其反向边所在的面连起来。

- 平面图转对偶图的方法。
- 大致想法是把一条无向边拆成两条有向边，然后确定每条有向边所在的面，并把一条边所在的面和其反向边所在的面连起来。
- 具体实现时先把每个点出发的所有边按照逆时针顺序排序，然后把所有边扫一遍，扫到一条边的时候把这条边所属面的所有边都遍历一遍，并确定这些边都属于那个面。

- 平面图转对偶图的方法。
- 大致想法是把一条无向边拆成两条有向边，然后确定每条有向边所在的面，并把一条边所在的面和其反向边所在的面连起来。
- 具体实现时先把每个点出发的所有边按照逆时针顺序排序，然后把所有边扫一遍，扫到一条边的时候把这条边所属面的所有边都遍历一遍，并确定这些边都属于那个面。
- 遍历一个面的所有边需要找到一条边的下一条边，当遍历一条边后，找到它反向边的顺时针方向的第一条边即可。

- 给定一张 n 个点 m 条边的平面图， k 次询问一个区域内所有面的面积平方之和与面积和的比值，以分数形式输出。
- 第 i 次询问的区域由 c_i 个点给出，保证在平面图上形成封闭多边形。
- $1 \leq n, k \leq 2 \times 10^5$ ，强制在线。

- 平面图转对偶图，然后找出一棵以无穷区域为根的生成树。

- 平面图转对偶图，然后找出一棵以无穷区域为根的生成树。
- 算出每棵子树的面积和以及面积平方和。

- 平面图转对偶图，然后找出一棵以无穷区域为根的生成树。
- 算出每棵子树的面积和以及面积平方和。
- 然后逆时针遍历边界上的所有边，分三种情况：

- 平面图转对偶图，然后找出一棵以无穷区域为根的生成树。
- 算出每棵子树的面积和以及面积平方和。
- 然后逆时针遍历边界上的所有边，分三种情况：
 1. 若其所属的面与其反向边所属的面不是父子关系，直接跳过。

- 平面图转对偶图，然后找出一棵以无穷区域为根的生成树。
- 算出每棵子树的面积和以及面积平方和。
- 然后逆时针遍历边界上的所有边，分三种情况：
 1. 若其所属的面与其反向边所属的面不是父子关系，直接跳过。
 2. 若其所属的面是其反向边所属的面的父亲，则减去儿子的子树和。

- 平面图转对偶图，然后找出一棵以无穷区域为根的生成树。
- 算出每棵子树的面积和以及面积平方和。
- 然后逆时针遍历边界上的所有边，分三种情况：
 1. 若其所属的面与其反向边所属的面不是父子关系，直接跳过。
 2. 若其所属的面是其反向边所属的面的父亲，则减去儿子的子树和。
 3. 若其所属的面是其反向边所属的面的儿子，则加上儿子的子树和。

- 平面图转对偶图，然后找出一棵以无穷区域为根的生成树。
- 算出每棵子树的面积和以及面积平方和。
- 然后逆时针遍历边界上的所有边，分三种情况：
 1. 若其所属的面与其反向边所属的面不是父子关系，直接跳过。
 2. 若其所属的面是其反向边所属的面的父亲，则减去儿子的子树和。
 3. 若其所属的面是其反向边所属的面的儿子，则加上儿子的子树和。
- 正确性画个图就很好理解了。

哈斯图

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。
- 偏序关系：对于二元关系 $R \subseteq S \times S$ ，如果 R 具有自反性、反对称性、传递性，那么 R 称为偏序关系。

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。
- 偏序关系：对于二元关系 $R \subseteq S \times S$ ，如果 R 具有自反性、反对称性、传递性，那么 R 称为偏序关系。
- 自反性： $\forall a \in S, a \leq a$ 。

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。
- 偏序关系：对于二元关系 $R \subseteq S \times S$ ，如果 R 具有自反性、反对称性、传递性，那么 R 称为偏序关系。
- 自反性： $\forall a \in S, a \leq a$ 。
- 反对称性： $\forall a, b \in S$ ，若 $a \leq b$ 且 $b \leq a$ ，则 $a = b$ 。

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。
- 偏序关系：对于二元关系 $R \subseteq S \times S$ ，如果 R 具有自反性、反对称性、传递性，那么 R 称为偏序关系。
- 自反性： $\forall a \in S, a \leq a$ 。
- 反对称性： $\forall a, b \in S$ ，若 $a \leq b$ 且 $b \leq a$ ，则 $a = b$ 。
- 传递性： $\forall a, b, c \in S$ ，若 $a \leq b$ 且 $b \leq c$ ，则 $a \leq c$ 。

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。
- 偏序关系：对于二元关系 $R \subseteq S \times S$ ，如果 R 具有自反性、反对称性、传递性，那么 R 称为偏序关系。
- 自反性： $\forall a \in S, a \leq a$ 。
- 反对称性： $\forall a, b \in S$ ，若 $a \leq b$ 且 $b \leq a$ ，则 $a = b$ 。
- 传递性： $\forall a, b, c \in S$ ，若 $a \leq b$ 且 $b \leq c$ ，则 $a \leq c$ 。
- 可比：若 $a, b \in S$ 满足 $a \leq b$ 或 $b \leq a$ ，则称 a, b 可比。

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。
- 偏序关系：对于二元关系 $R \subseteq S \times S$ ，如果 R 具有自反性、反对称性、传递性，那么 R 称为偏序关系。
- 自反性： $\forall a \in S, a \leq a$ 。
- 反对称性： $\forall a, b \in S$ ，若 $a \leq b$ 且 $b \leq a$ ，则 $a = b$ 。
- 传递性： $\forall a, b, c \in S$ ，若 $a \leq b$ 且 $b \leq c$ ，则 $a \leq c$ 。
- 可比：若 $a, b \in S$ 满足 $a \leq b$ 或 $b \leq a$ ，则称 a, b 可比。
- 哈斯图：对于元素 x ，如果 $x < y$ 且不存在 z 使得 $x < z < y$ ，那么 y 就是 x 的覆盖元素，在哈斯图中连出一条有向边 (x, y) 。通过覆盖关系生成的图就是哈斯图。

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。
- 偏序关系：对于二元关系 $R \subseteq S \times S$ ，如果 R 具有自反性、反对称性、传递性，那么 R 称为偏序关系。
- 自反性： $\forall a \in S, a \leq a$ 。
- 反对称性： $\forall a, b \in S$ ，若 $a \leq b$ 且 $b \leq a$ ，则 $a = b$ 。
- 传递性： $\forall a, b, c \in S$ ，若 $a \leq b$ 且 $b \leq c$ ，则 $a \leq c$ 。
- 可比：若 $a, b \in S$ 满足 $a \leq b$ 或 $b \leq a$ ，则称 a, b 可比。
- 哈斯图：对于元素 x ，如果 $x < y$ 且不存在 z 使得 $x < z < y$ ，那么 y 就是 x 的覆盖元素，在哈斯图中连出一条有向边 (x, y) 。通过覆盖关系生成的图就是哈斯图。
- 链：设 $C \subseteq S$ ，若 C 中元素互相可比，则称 C 是偏序集 (S, R) 的链。

定义

- 偏序集：偏序集由集合 S 及其偏序关系 R 构成，记作 (S, R) 。
- 偏序关系：对于二元关系 $R \subseteq S \times S$ ，如果 R 具有自反性、反对称性、传递性，那么 R 称为偏序关系。
- 自反性： $\forall a \in S, a \leq a$ 。
- 反对称性： $\forall a, b \in S$ ，若 $a \leq b$ 且 $b \leq a$ ，则 $a = b$ 。
- 传递性： $\forall a, b, c \in S$ ，若 $a \leq b$ 且 $b \leq c$ ，则 $a \leq c$ 。
- 可比：若 $a, b \in S$ 满足 $a \leq b$ 或 $b \leq a$ ，则称 a, b 可比。
- 哈斯图：对于元素 x ，如果 $x < y$ 且不存在 z 使得 $x < z < y$ ，那么 y 就是 x 的覆盖元素，在哈斯图中连出一条有向边 (x, y) 。通过覆盖关系生成的图就是哈斯图。
- 链：设 $C \subseteq S$ ，若 C 中元素互相可比，则称 C 是偏序集 (S, R) 的链。
- 反链：设 $C \subseteq S$ ，若 C 中元素互相不可比，则称 C 是偏序集 (S, R) 的反链。

- 由于偏序关系具有反对称性，因此哈斯图中不会出现回路。

- 由于偏序关系具有反对称性，因此哈斯图中不会出现回路。
- 因此哈斯图是一个有向无环图。

- 由于偏序关系具有反对称性，因此哈斯图中不会出现回路。
- 因此哈斯图是一个有向无环图。
- 拓扑排序本质就是从偏序构造一个相容的全序。

定理 10.3.1 (Dilworth 定理)

对于任意有限偏序集，其最大反链中元素的数目必等于最小链划分中链的数目。

定理 10.3.1 (Dilworth 定理)

对于任意有限偏序集，其最大反链中元素的数目必等于最小链划分中链的数目。

引理 10.3.1

最小链覆盖大小不小于最长反链大小。

定理 10.3.1 (Dilworth 定理)

对于任意有限偏序集，其最大反链中元素的数目必等于最小链划分中链的数目。

引理 10.3.1

最小链覆盖大小不小于最长反链大小。

- 此引理可以使用鸽巢原理和反证法简单证明。

证明.

- 考虑使用数学归纳法。



证明.

- 考虑使用数学归纳法。
- 当 $|S| = 1$ 时显然。



证明.

- 考虑使用数学归纳法。
- 当 $|S| = 1$ 时显然。
- 当 $|S| > 1$ 时，考虑拓扑序最大的元素 x 以及 $S/\{x\}$ 。



证明.

- 考虑使用数学归纳法。
- 当 $|S| = 1$ 时显然。
- 当 $|S| > 1$ 时，考虑拓扑序最大的元素 x 以及 $S/\{x\}$ 。
- S 的最小链覆盖大于 $S/\{x\}$ 当且仅当 x 与所有 $S/\{x\}$ 中的元素均不可比。



证明.

- 考虑使用数学归纳法。
- 当 $|S| = 1$ 时显然。
- 当 $|S| > 1$ 时，考虑拓扑序最大的元素 x 以及 $S/\{x\}$ 。
- S 的最小链覆盖大于 $S/\{x\}$ 当且仅当 x 与所有 $S/\{x\}$ 中的元素均不可比。
- 若最小链覆盖大小变化， x 一定可以直接加入最长反链。



证明.

- 考虑使用数学归纳法。
- 当 $|S| = 1$ 时显然。
- 当 $|S| > 1$ 时，考虑拓扑序最大的元素 x 以及 $S/\{x\}$ 。
- S 的最小链覆盖大于 $S/\{x\}$ 当且仅当 x 与所有 $S/\{x\}$ 中的元素均不可比。
- 若最小链覆盖大小变化， x 一定可以直接加入最长反链。
- 因此若最小链覆盖大小变化，则最长反链大小也一定变化。



证明.

- 考虑使用数学归纳法。
- 当 $|S| = 1$ 时显然。
- 当 $|S| > 1$ 时，考虑拓扑序最大的元素 x 以及 $S/\{x\}$ 。
- S 的最小链覆盖大于 $S/\{x\}$ 当且仅当 x 与所有 $S/\{x\}$ 中的元素均不可比。
- 若最小链覆盖大小变化， x 一定可以直接加入最长反链。
- 因此若最小链覆盖大小变化，则最长反链大小也一定变化。
- 所以若最长反链大小不变，则最小链覆盖大小也一定不变。



证明.

- 考虑使用数学归纳法。
- 当 $|S| = 1$ 时显然。
- 当 $|S| > 1$ 时，考虑拓扑序最大的元素 x 以及 $S/\{x\}$ 。
- S 的最小链覆盖大于 $S/\{x\}$ 当且仅当 x 与所有 $S/\{x\}$ 中的元素均不可比。
- 若最小链覆盖大小变化， x 一定可以直接加入最长反链。
- 因此若最小链覆盖大小变化，则最长反链大小也一定变化。
- 所以若最长反链大小不变，则最小链覆盖大小也一定不变。
- 剩余情况由引理 9.3.1 可以得知不可能存在，因此定理得证。



- 对有限二维偏序集求最长链及其最小链覆盖。

- 对有限二维偏序集求最长链及其最小链覆盖。
- 最长链即最长上升子序列；最小链覆盖即最长反链，对应最长下降子序列。

环计数问题

- 将所有点按度数排序，每条边定向为从度数大的连向度数小的，不难发现这就是一个拓扑序。

三元环计数

- 将所有点按度数排序，每条边定向为从度数大的连向度数小的，不难发现这就是一个拓扑序。
- 可以证明此时每个点的入边数不超过 $O(\sqrt{m})$ 。

三元环计数

- 将所有点按度数排序，每条边定向为从度数大的连向度数小的，不难发现这就是一个拓扑序。
- 可以证明此时每个点的入边数不超过 $O(\sqrt{m})$ 。
- 证明：度数小于 $O(\sqrt{m})$ 的点显然；度数大于 $O(\sqrt{m})$ 的点数量不会超过 $O(\sqrt{m})$ ，又由于只有度数更大的点会连向它，因此入边数不会超过 $O(\sqrt{m})$ 。

三元环计数

- 将所有点按度数排序，每条边定向为从度数大的连向度数小的，不难发现这就是一个拓扑序。
- 可以证明此时每个点的入边数不超过 $O(\sqrt{m})$ 。
- 证明：度数小于 $O(\sqrt{m})$ 的点显然；度数大于 $O(\sqrt{m})$ 的点数量不会超过 $O(\sqrt{m})$ ，又由于只有度数更大的点会连向它，因此入边数不会超过 $O(\sqrt{m})$ 。
- 枚举每个点 a 的每条出边 (a, b) ，再枚举 b 的每条出边 (b, c) ，若存在边 (a, c) ，则说明找到了一个三元环 (a, b, c) 。

三元环计数

- 将所有点按度数排序，每条边定向为从度数大的连向度数小的，不难发现这就是一个拓扑序。
- 可以证明此时每个点的入边数不超过 $O(\sqrt{m})$ 。
- 证明：度数小于 $O(\sqrt{m})$ 的点显然；度数大于 $O(\sqrt{m})$ 的点数量不会超过 $O(\sqrt{m})$ ，又由于只有度数更大的点会连向它，因此入边数不会超过 $O(\sqrt{m})$ 。
- 枚举每个点 a 的每条出边 (a, b) ，再枚举 b 的每条出边 (b, c) ，若存在边 (a, c) ，则说明找到了一个三元环 (a, b, c) 。
- 由于每个点至多被访问 $O(\sqrt{m})$ 次，因此时间复杂度 $O(m\sqrt{m})$ 。

三元环计数

- 将所有点按度数排序，每条边定向为从度数大的连向度数小的，不难发现这就是一个拓扑序。
- 可以证明此时每个点的入边数不超过 $O(\sqrt{m})$ 。
- 证明：度数小于 $O(\sqrt{m})$ 的点显然；度数大于 $O(\sqrt{m})$ 的点数量不会超过 $O(\sqrt{m})$ ，又由于只有度数更大的点会连向它，因此入边数不会超过 $O(\sqrt{m})$ 。
- 枚举每个点 a 的每条出边 (a, b) ，再枚举 b 的每条出边 (b, c) ，若存在边 (a, c) ，则说明找到了一个三元环 (a, b, c) 。
- 由于每个点至多被访问 $O(\sqrt{m})$ 次，因此时间复杂度 $O(m\sqrt{m})$ 。
- 实际上这是在枚举每个三元环，因此我们也同时证明了三元环的数量是不超过 $O(m\sqrt{m})$ 的。

- 用跟三元环计数同样的方法将图转化为有向无环图。

四元环计数

- 用跟三元环计数的方法将图转化为有向无环图。
- 枚举每个点 a 的每条出边 (a, b) ，再枚举 b 的每条边 (b, c) ，答案加上 c 的计数器并对 c 的计数器加一。

四元环计数

- 用跟三元环计数的方法将图转化为有向无环图。
- 枚举每个点 a 的每条出边 (a, b) ，再枚举 b 的每条边 (b, c) ，答案加上 c 的计数器并对 c 的计数器加一。
- 枚举每个点后需要把计数器回滚。

四元环计数

- 用跟三元环计数同样的方法将图转化为有向无环图。
- 枚举每个点 a 的每条出边 (a, b) ，再枚举 b 的每条边 (b, c) ，答案加上 c 的计数器并对 c 的计数器加一。
- 枚举每个点后需要把计数器回滚。
- 时间复杂度 $O(m\sqrt{m})$ 。

最大团 / 最大独立集搜索

- 给定一张图，求最大团 / 最大独立集 / 极大团 / 极大独立集。

- Bron-Kerbosch 的大致思路是维护三个点集 R, P, X , 其中 R 是当前搜到的团的集合, P 是等待搜索的点的集合, 满足 P 中的点与 R 中任意一个点都有边, X 是回溯时从团里扔出来的点的集合, 满足 X 中的点与 R 中任意一个点都有边。

- Bron-Kerbosch 的大致思路是维护三个点集 R, P, X , 其中 R 是当前搜到的团的集合, P 是等待搜索的点的集合, 满足 P 中的点与 R 中任意一个点都有边, X 是回溯时从团里扔出来的点的集合, 满足 X 中的点与 R 中任意一个点都有边。
- 设 $N(u)$ 表示所有与 u 有边的点的集合。

- Bron-Kerbosch 的大致思路是维护三个点集 R, P, X , 其中 R 是当前搜到的团的集合, P 是等待搜索的点的集合, 满足 P 中的点与 R 中任意一个点都有边, X 是回溯时从团里扔出来的点的集合, 满足 X 中的点与 R 中任意一个点都有边。
- 设 $N(u)$ 表示所有与 u 有边的点的集合。
- 初始时令 $P = V, R = X = \emptyset$ 。

- Bron-Kerbosch 的大致思路是维护三个点集 R, P, X , 其中 R 是当前搜到的团的集合, P 是等待搜索的点的集合, 满足 P 中的点与 R 中任意一个点都有边, X 是回溯时从团里扔出来的点的集合, 满足 X 中的点与 R 中任意一个点都有边。
- 设 $N(u)$ 表示所有与 u 有边的点的集合。
- 初始时令 $P = V, R = X = \emptyset$ 。
- 若当前 $P = \emptyset$:

- Bron-Kerbosch 的大致思路是维护三个点集 R, P, X , 其中 R 是当前搜到的团的集合, P 是等待搜索的点的集合, 满足 P 中的点与 R 中任意一个点都有边, X 是回溯时从团里扔出来的点的集合, 满足 X 中的点与 R 中任意一个点都有边。
- 设 $N(u)$ 表示所有与 u 有边的点的集合。
- 初始时令 $P = V, R = X = \emptyset$ 。
- 若当前 $P = \emptyset$:
 - 若 $X = \emptyset$, 则 R 是一个极大团。

- Bron-Kerbosch 的大致思路是维护三个点集 R, P, X , 其中 R 是当前搜到的团的集合, P 是等待搜索的点的集合, 满足 P 中的点与 R 中任意一个点都有边, X 是回溯时从团里扔出来的点的集合, 满足 X 中的点与 R 中任意一个点都有边。
- 设 $N(u)$ 表示所有与 u 有边的点的集合。
- 初始时令 $P = V, R = X = \emptyset$ 。
- 若当前 $P = \emptyset$:
 - 若 $X = \emptyset$, 则 R 是一个极大团。
 - 否则, 回溯。

- Bron-Kerbosch 的大致思路是维护三个点集 R, P, X , 其中 R 是当前搜到的团的集合, P 是等待搜索的点的集合, 满足 P 中的点与 R 中任意一个点都有边, X 是回溯时从团里扔出来的点的集合, 满足 X 中的点与 R 中任意一个点都有边。
- 设 $N(u)$ 表示所有与 u 有边的点的集合。
- 初始时令 $P = V, R = X = \emptyset$ 。
- 若当前 $P = \emptyset$:
 - 若 $X = \emptyset$, 则 R 是一个极大团。
 - 否则, 回溯。
- 每次从 P 中任意取出一个点 v , 令 $R' = R \cup \{v\}, P' = P \cap N(v), X' = X \cap N(v)$, 对 R', P', X' 继续递归搜索。

- Bron-Kerbosch 的大致思路是维护三个点集 R, P, X , 其中 R 是当前搜到的团的集合, P 是等待搜索的点的集合, 满足 P 中的点与 R 中任意一个点都有边, X 是回溯时从团里扔出来的点的集合, 满足 X 中的点与 R 中任意一个点都有边。
- 设 $N(u)$ 表示所有与 u 有边的点的集合。
- 初始时令 $P = V, R = X = \emptyset$ 。
- 若当前 $P = \emptyset$:
 - 若 $X = \emptyset$, 则 R 是一个极大团。
 - 否则, 回溯。
- 每次从 P 中任意取出一个点 v , 令 $R' = R \cup \{v\}, P' = P \cap N(v), X' = X \cap N(v)$, 对 R', P', X' 继续递归搜索。
- 搜索结束后从 P 中删除 v , 将其加入 X 中。

- 伪代码如下：

```
1 | function BronKerbosch( $R, P, X$ ) :  
2 |     if  $P = \emptyset \wedge X = \emptyset$  then  
3 |         report  $R$  as a maximal clique.  
4 |     for each  $v \in P$  do  
5 |         BronKerbosch( $R \cup \{v\}, P \cap N(v), X \cap N(v)$ )  
6 |          $P \leftarrow P \setminus \{v\}$   
7 |          $X \leftarrow X \cup \{v\}$ 
```

- 优化 1: 对于任意一个点 u , 必然要满足 u 最后在 R 中或最后 R 中存在一个点 v 使得 v 与 u 之间没有边, 这个团才有可能对答案产生贡献。

```
1 function BronKerbosch( $R, P, X$ ) :  
2     if  $P = \emptyset \wedge X = \emptyset$  then  
3         report  $R$  as a maximal clique.  
4     choose a vertex  $u \in P \cup X$   
5     for each  $v \in P \setminus N(u)$  do  
6         BronKerbosch( $R \cup \{v\}, P \cap N(v), X \cap N(v)$ )  
7          $P \leftarrow P \setminus \{v\}$   
8          $X \leftarrow X \cup \{v\}$ 
```

- 优化 2: 我们每次取出度数最小的点, 将其加到队列末尾然后删掉这个点, 这样给点排了个序, 称之为 degeneracy ordering, 在搜索的第一层按这个顺序搜索。

```
1 | function solve( $G$ ) :  
2 |    $P = V, X = \emptyset$   
5 |   for each  $v$  in a degeneracy ordering of  $G$  do  
6 |     BronKerbosch( $\{v\}, P \cap N(v), X \cap N(v)$ )  
7 |      $P \leftarrow P \setminus \{v\}$   
8 |      $X \leftarrow X \cup \{v\}$ 
```


- 可以证明, 任意 n 个顶点的图至多只有 $3^{n/3}$ 个极大团。

- 可以证明，任意 n 个顶点的图至多只有 $3^{n/3}$ 个极大团。
- 使用了优化 1 的 Bron-Kerbosch 的时间复杂度为 $O(3^{n/3})$ 。

- 可以证明，任意 n 个顶点的图至多只有 $3^{n/3}$ 个极大团。
- 使用了优化 1 的 Bron-Kerbosch 的时间复杂度为 $O(3^{n/3})$ 。
- 对于稀疏图存在更紧的界。使用了优化 2 的 Bron-Kerbosch 的时间复杂度为 $O(dn^{d/3})$ ，其中 d 为图的 degeneracy。

- 可以证明，任意 n 个顶点的图至多只有 $3^{n/3}$ 个极大团。
- 使用了优化 1 的 Bron-Kerbosch 的时间复杂度为 $O(3^{n/3})$ 。
- 对于稀疏图存在更紧的界。使用了优化 2 的 Bron-Kerbosch 的时间复杂度为 $O(dn^{d/3})$ ，其中 d 为图的 degeneracy。
- 图的 degeneracy 被定义为最小的 d 使得其任意导出子图都至少存在一个点的度数 $\leq d$ 。一张 degeneracy 为 d 的图至多存在 $(n - d)3^{d/3}$ 个极大团。

- 可以证明，任意 n 个顶点的图至多只有 $3^{n/3}$ 个极大团。
- 使用了优化 1 的 Bron-Kerbosch 的时间复杂度为 $O(3^{n/3})$ 。
- 对于稀疏图存在更紧的界。使用了优化 2 的 Bron-Kerbosch 的时间复杂度为 $O(dn^{d/3})$ ，其中 d 为图的 degeneracy。
- 图的 degeneracy 被定义为最小的 d 使得其任意导出子图都至少存在一个点的度数 $\leq d$ 。一张 degeneracy 为 d 的图至多存在 $(n - d)3^{d/3}$ 个极大团。
- 具体细节可以参考 Bron-Kerbosch algorithm - Wikipedia。

- 分支算法是一类对可行解进行穷举的算法，主要有以下两条规则：

- 分支算法是一类对可行解进行穷举的算法，主要有以下两条规则：
- reduction rule: 用于简化问题或停止算法。

- 分支算法是一类对可行解进行穷举的算法，主要有以下两条规则：
- reduction rule: 用于简化问题或停止算法。
- branching rule: 递归求解更小的情况。

- Branching 算法的上界分析需要一些假设：

- Branching 算法的上界分析需要一些假设：
- 在一些简单的情况下可以做到多项式时间。

- Branching 算法的上界分析需要一些假设：
- 在一些简单的情况下可以做到多项式时间。
- 对问题递归会变成更简单的情况。

- Branching 算法的上界分析需要一些假设：
- 在一些简单的情况下可以做到多项式时间。
- 对问题递归会变成更简单的情况。
- 基于这些假设，我们可以简单地把复杂度视作搜索树结点个数 $\times$ 一个多项式。

- 我们用一些参数来描述问题规模，如图的结点，称其为 instance。

上界分析

- 我们用一些参数来描述问题规模，如图的结点，称其为 instance。
- 设 $T(n)$ 为 instance 数量为 n 的一个问题的搜索树叶子结点个数。

上界分析

- 我们用一些参数来描述问题规模，如图的结点，称其为 instance。
- 设 $T(n)$ 为 instance 数量为 n 的一个问题的搜索树叶子结点个数。
- 对于一个 branching rule b ，设 $r \geq 2, \forall 1 \leq i \leq r, t_i > 0$ ，规则 b 将 instance 分成 r 个子问题，且每个子问题的 instance 数量至多为 $n - t_i$ 。

上界分析

- 我们用一些参数来描述问题规模，如图的结点，称其为 instance。
- 设 $T(n)$ 为 instance 数量为 n 的一个问题的搜索树叶子结点个数。
- 对于一个 branching rule b ，设 $r \geq 2, \forall 1 \leq i \leq r, t_i > 0$ ，规则 b 将 instance 分成 r 个子问题，且每个子问题的 instance 数量至多为 $n - t_i$ 。
- 称向量 $b = (t_1, t_2, \dots, t_r)$ 为 branching vector，根据递归树的结构，可以得到：

$$T(n) \leq \sum_{i=1}^r T(n - t_i)$$

上界分析

- 我们用一些参数来描述问题规模，如图的结点，称其为 instance。
- 设 $T(n)$ 为 instance 数量为 n 的一个问题的搜索树叶子结点个数。
- 对于一个 branching rule b ，设 $r \geq 2, \forall 1 \leq i \leq r, t_i > 0$ ，规则 b 将 instance 分成 r 个子问题，且每个子问题的 instance 数量至多为 $n - t_i$ 。
- 称向量 $b = (t_1, t_2, \dots, t_r)$ 为 branching vector，根据递归树的结构，可以得到：

$$T(n) \leq \sum_{i=1}^r T(n - t_i)$$

- 考虑设等式 $T^*(n) = \sum_{i=1}^r T^*(n - t_i)$ ，解方程 $x^n - \sum_{i=1}^r x^{n-t_i} = 0$ ，设得到的最大正根为 α ，可以认为 $T(n) \leq T^*(n) = O(\alpha^n)$ 。

上界分析

- 我们用一些参数来描述问题规模，如图的结点，称其为 instance。
- 设 $T(n)$ 为 instance 数量为 n 的一个问题的搜索树叶子结点个数。
- 对于一个 branching rule b ，设 $r \geq 2, \forall 1 \leq i \leq r, t_i > 0$ ，规则 b 将 instance 分成 r 个子问题，且每个子问题的 instance 数量至多为 $n - t_i$ 。
- 称向量 $b = (t_1, t_2, \dots, t_r)$ 为 branching vector，根据递归树的结构，可以得到：

$$T(n) \leq \sum_{i=1}^r T(n - t_i)$$

- 考虑设等式 $T^*(n) = \sum_{i=1}^r T^*(n - t_i)$ ，解方程 $x^n - \sum_{i=1}^r x^{n-t_i} = 0$ ，设得到的最大正根为 α ，可以认为 $T(n) \leq T^*(n) = O(\alpha^n)$ 。
- 称 α 为 b 的 branching factor，记作 $\tau(t_1, t_2, \dots, t_r)$ 。

最大独立集

- 对于最大独立集，我们可以设计以下分支算法：

```
1  function MIS( $G$ ) :
2      if  $\exists v \in V$  with  $d(v) = 0$  then
3          return  $1 + MIS(G \setminus \{v\})$ 
4      if  $\exists v \in V$  with  $d(v) = 1$  then
5          return  $1 + MIS(G \setminus (\{v\} \cup N(v)))$ 
6      if  $\max_v d(v) \geq 3$  then
7          choose a vertex  $v$  of maximum degree in  $G$ .
8          return  $\max\{1 + MIS(G \setminus (\{v\} \cup N(v))), MIS(G \setminus \{v\})\}$ 
9      if  $\max_v d(v) \leq 2$  then
10         compute ans of  $G$  in polynomial time
11         return ans
```

- 显然这个算法只有在第三个 if 中会有分支，其 branching vector 为 $(4, 1)$ ，可以算出 $\alpha \approx 1.3803$ 。

- 显然这个算法只有在第三个 if 中会有分支，其 branching vector 为 $(4, 1)$ ，可以算出 $\alpha \approx 1.3803$ 。
- 这样我们就得到了一个比较宽的复杂度上界为 $O(1.3803^n)$ 。

- M&C 的基本思想是，通过给有不同性质的元素赋不同的权值，使分支算法的复杂度可以分析到一个更好的界。

- M&C 的基本思想是，通过给有不同性质的元素赋不同的权值，使分支算法的复杂度可以分析到一个更好的界。
- 对于上面的算法，我们考虑每个点的度数。

- M&C 的基本思想是，通过给有不同性质的元素赋不同的权值，使分支算法的复杂度可以分析到一个更好的界。
- 对于上面的算法，我们考虑每个点的度数。
- 对于度数为 0 或 1 的点，它们不会在分支中出现，可以将其权值设为 0。

- M&C 的基本思想是，通过给有不同性质的元素赋不同的权值，使分支算法的复杂度可以分析到一个更好的界。
- 对于上面的算法，我们考虑每个点的度数。
- 对于度数为 0 或 1 的点，它们不会在分支中出现，可以将其权值设为 0。
- 我们不妨直接令度数为 2 的点的权值为 0.5，度数大于等于 3 的点的权值为 1，则现在的 instance 数量为 $k = 0.5n_2 + n_{\geq 3}$ ，其中 n_2 为度数为 2 的点的个数， $n_{\geq 3}$ 为度数大于等于 3 的点的个数。

- M&C 的基本思想是，通过给有不同性质的元素赋不同的权值，使分支算法的复杂度可以分析到一个更好的界。
- 对于上面的算法，我们考虑每个点的度数。
- 对于度数为 0 或 1 的点，它们不会在分支中出现，可以将其权值设为 0。
- 我们不妨直接令度数为 2 的点的权值为 0.5，度数大于等于 3 的点的权值为 1，则现在的 instance 数量为 $k = 0.5n_2 + n_{\geq 3}$ ，其中 n_2 为度数为 2 的点的个数， $n_{\geq 3}$ 为度数大于等于 3 的点的个数。
- 此时我们考虑分支对 k 的影响。

- 对于 $d(v) > 3$, 仍沿用原本的分析, 其 branching vector 为 $(5, 1)$, $\tau(5, 1) \approx 1.3247$ 。

- 对于 $d(v) > 3$, 仍沿用原本的分析, 其 branching vector 为 $(5, 1)$, $\tau(5, 1) \approx 1.3247$ 。
- 对于 $d(v) = 3$, 若将 v 加入最大独立集, 则 k 会减少 1, 且其每个邻居至少有 0.5 的权值, 因此 k 至少减少 2.5。

- 对于 $d(v) > 3$, 仍沿用原本的分析, 其 branching vector 为 $(5, 1)$, $\tau(5, 1) \approx 1.3247$ 。
- 对于 $d(v) = 3$, 若将 v 加入最大独立集, 则 k 会减少 1, 且其每个邻居至少有 0.5 的权值, 因此 k 至少减少 2.5。
- 若不将 v 加入独立集, 则 k 会减少 1, 且对于其每个邻居, 度数都会减少 1, 如果是 3 变 2 则 k 会减少 0.5, 如果是 2 变 1 则 k 也会减少 0.5, 因此 k 至少会减少 2.5。

- 对于 $d(v) > 3$, 仍沿用原本的分析, 其 branching vector 为 $(5, 1)$, $\tau(5, 1) \approx 1.3247$ 。
- 对于 $d(v) = 3$, 若将 v 加入最大独立集, 则 k 会减少 1, 且其每个邻居至少有 0.5 的权值, 因此 k 至少减少 2.5。
- 若不将 v 加入独立集, 则 k 会减少 1, 且对于其每个邻居, 度数都会减少 1, 如果是 3 变 2 则 k 会减少 0.5, 如果是 2 变 1 则 k 也会减少 0.5, 因此 k 至少会减少 2.5。
- 此时的 branching vector 为 $(2.5, 2.5)$, $\tau(2.5, 2.5) \approx 1.3195$ 。

- 对于 $d(v) > 3$ ，仍沿用原本的分析，其 branching vector 为 $(5, 1)$ ， $\tau(5, 1) \approx 1.3247$ 。
- 对于 $d(v) = 3$ ，若将 v 加入最大独立集，则 k 会减少 1，且其每个邻居至少有 0.5 的权值，因此 k 至少减少 2.5。
- 若不将 v 加入独立集，则 k 会减少 1，且对于其每个邻居，度数都会减少 1，如果是 3 变 2 则 k 会减少 0.5，如果是 2 变 1 则 k 也会减少 0.5，因此 k 至少会减少 2.5。
- 此时的 branching vector 为 $(2.5, 2.5)$ ， $\tau(2.5, 2.5) \approx 1.3195$ 。
- 因此可以得到更紧的时间复杂度为 $O(1.3247^n)$ 。

- 对于 $d(v) > 3$, 仍沿用原本的分析, 其 branching vector 为 $(5, 1)$, $\tau(5, 1) \approx 1.3247$ 。
- 对于 $d(v) = 3$, 若将 v 加入最大独立集, 则 k 会减少 1, 且其每个邻居至少有 0.5 的权值, 因此 k 至少减少 2.5。
- 若不将 v 加入独立集, 则 k 会减少 1, 且对于其每个邻居, 度数都会减少 1, 如果是 3 变 2 则 k 会减少 0.5, 如果是 2 变 1 则 k 也会减少 0.5, 因此 k 至少会减少 2.5。
- 此时的 branching vector 为 $(2.5, 2.5)$, $\tau(2.5, 2.5) \approx 1.3195$ 。
- 因此可以得到更紧的时间复杂度为 $O(1.3247^n)$ 。
- 可以证明这样赋权是最优的。

- 相较于 Bron-Kerbosch, 分支算法复杂度更加优秀, 原因是没有枚举所有的极大独立集。因此它也无法对极大独立集进行计数。

- 相较于 Bron-Kerbosch, 分支算法复杂度更加优秀, 原因是没有枚举所有的极大独立集。因此它也无法对极大独立集进行计数。
- 关于分支算法和 Measure and Conquer 的分析方法由于时间有限因此不能详细讲述, 感兴趣的同学可以参考以下文章:

- 相较于 Bron-Kerbosch，分支算法复杂度更加优秀，原因是没有枚举所有的极大独立集。因此它也无法对极大独立集进行计数。
- 关于分支算法和 Measure and Conquer 的分析方法由于时间有限因此不能详细讲述，感兴趣的同学可以参考以下文章：
 - <https://baoduoxu.github.io/2023/08/17/exact-exponential-algorithm/exact-algorithm-branching/>

- 相较于 Bron-Kerbosch, 分支算法复杂度更加优秀, 原因是没有枚举所有的极大独立集。因此它也无法对极大独立集进行计数。
- 关于分支算法和 Measure and Conquer 的分析方法由于时间有限因此不能详细讲述, 感兴趣的同学可以参考以下文章:
 - <https://baoduoxu.github.io/2023/08/17/exact-exponential-algorithm/exact-algorithm-branching/>
 - <https://baoduoxu.github.io/2023/09/04/exact-exponential-algorithm/exact-algorithm-measure-and-conquer/>

- 相较于 Bron-Kerbosch，分支算法复杂度更加优秀，原因是没有枚举所有的极大独立集。因此它也无法对极大独立集进行计数。
- 关于分支算法和 Measure and Conquer 的分析方法由于时间有限因此不能详细讲述，感兴趣的同学可以参考以下文章：
 - <https://baoduoxu.github.io/2023/08/17/exact-exponential-algorithm/exact-algorithm-branching/>
 - <https://baoduoxu.github.io/2023/09/04/exact-exponential-algorithm/exact-algorithm-measure-and-conquer/>
 - <https://zhuanlan.zhihu.com/p/668645340>

耳分解与双极定向

- 定义图 $G = (V, E)$ 的一个子图 $G' = (V', E')$ 的耳是一个顶点序列 $x_1, x_2, x_3, \dots, x_k$, 满足任意相邻两点间有边, 且 $x_1, x_k \in V'$, $x_2, x_3, \dots, x_{k-1} \in V \setminus V'$, 且 $x_2, x_3, \dots, x_{k-1}$ 互不相同。

- 定义图 $G = (V, E)$ 的一个子图 $G' = (V', E')$ 的耳是一个顶点序列 $x_1, x_2, x_3, \dots, x_k$, 满足任意相邻两点间有边, 且 $x_1, x_k \in V'$, $x_2, x_3, \dots, x_{k-1} \in V \setminus V'$, 且 $x_2, x_3, \dots, x_{k-1}$ 互不相同。
- 若 $x_1 \neq x_k$, 则称其为开耳。

- 定义图 $G = (V, E)$ 的一个子图 $G' = (V', E')$ 的耳是一个顶点序列 $x_1, x_2, x_3, \dots, x_k$, 满足任意相邻两点间有边, 且 $x_1, x_k \in V'$, $x_2, x_3, \dots, x_{k-1} \in V \setminus V'$, 且 $x_2, x_3, \dots, x_{k-1}$ 互不相同。
- 若 $x_1 \neq x_k$, 则称其为开耳。
- 耳分解是将图 G 分解为一个子图序列 $(G_0, G_1, \dots, G_k)$, 满足 G_0 是一个环, $\forall 0 \leq i < k, G_i \subset G_{i+1}$, $G_K = G$, 且 $\forall 0 \leq i < k, G_{i+1} \setminus G_i$ 必须是一个耳 (去掉首尾, 首尾在 G_i 中)。

- 定义图 $G = (V, E)$ 的一个子图 $G' = (V', E')$ 的耳是一个顶点序列 $x_1, x_2, x_3, \dots, x_k$, 满足任意相邻两点间有边, 且 $x_1, x_k \in V'$, $x_2, x_3, \dots, x_{k-1} \in V \setminus V'$, 且 $x_2, x_3, \dots, x_{k-1}$ 互不相同。
- 若 $x_1 \neq x_k$, 则称其为开耳。
- 耳分解是将图 G 分解为一个子图序列 $(G_0, G_1, \dots, G_k)$, 满足 G_0 是一个环, $\forall 0 \leq i < k, G_i \subset G_{i+1}$, $G_K = G$, 且 $\forall 0 \leq i < k, G_{i+1} \setminus G_i$ 必须是一个耳 (去掉首尾, 首尾在 G_i 中)。
- 若这些耳都是开耳, 则称其为开耳分解。

定理 13.0.1

- 一张图边双连通 $\iff$ 这张图存在耳分解。
- 一张图点双连通 $\iff$ 这张图存在开耳分解。

耳分解与双连通性

证明.

考虑对加入耳的过程归纳，容易证明必要性。

接下来证充分性，考虑对 dfs 生成树构造一组耳分解 / 开耳分解。

首先任意选一个从根开始仅包含一条非树边的环作为 G_0 。

之后一直保持当前子图在树上是一个包含根的连通块。

如果当前子图的点集不是全集，则找到一个点 x 使得 x 不在子图中且 fa_x 在子图中。

考虑找出一条覆盖 (fa_x, x) 的返祖边 (u, v) ，对于边双连通图总是存在这样一条边，然后将 $fa_x \rightarrow x \rightarrow \cdots \rightarrow v \rightarrow u$ 作为一个耳加入耳分解中。

特别地，对于点双连通图，一定存在一条返祖边 (u, v) 使得 u 是 fa_x 的祖先，因此每次加入的耳都是开耳。

如果当前子图的点集已经是全集了，就将每条边都作为一个耳加入耳分解即可。 □

- 考虑这样一个问题：给定一张无向图，将其定向成强连通图。

- 考虑这样一个问题：给定一张无向图，将其定向成强连通图。
- 容易发现这张图首先必须是边双连通的，否则无解。

- 考虑这样一个问题：给定一张无向图，将其定向成强连通图。
- 容易发现这张图首先必须是边双连通的，否则无解。
- 然后对于边双连通图，可以将其耳分解，然后每次将耳任意定向成一条有向路径，容易发现始终保持强连通性。

- 考虑这样一个问题：给定一张无向图，将其定向成强连通图。
- 容易发现这张图首先必须是边双连通的，否则无解。
- 然后对于边双连通图，可以将其耳分解，然后每次将耳任意定向成一条有向路径，容易发现始终保持强连通性。
- 事实上，对于强连通图，可以仿照无向图的定义给出有向图中耳的定义。同样的，仿照无向图中的构造方法，可以给出有向图的耳分解方式。

- 对于一张无向图，要将其每条边定向，使得其成为一张 DAG，且仅有一个点 s 没有入度，仅有一个点 t 没有出度。

- 对于一张无向图，要将其每条边定向，使得其成为一张 DAG，且仅有一个点 s 没有入度，仅有一个点 t 没有出度。
- s, t 给定。

定理 13.0.2

一张无向图能双极定向当且仅当连接边 (s, t) 后图点双连通。

定理 13.0.2

一张无向图能双极定向当且仅当连接边 (s, t) 后图点双连通。

证明.

考虑点双连通的等价条件，即存在开耳分解。

必要性：首先令 G_0 是包含 (s, t) 的环，去掉这条边，剩下的从 s 到 t 定向。之后没增加一个耳，比较其首尾的偏序关系然后将其定向，容易发现这样构造一直满足双极定向的条件。

充分性：考虑反证，如果不是点双那必然存在割点，显然 s, t 不会是割点。删除割点后会形成一个包含 s, t 的连通块和若干其他连通块。考虑对割点所连的边定向，发现要么无法保证无环，要么无法保证双极性。 $\square$

- 一张图可以双极定向等价于存在一个点的排列 $p_1, p_2, \dots, p_n$, 其中 $p_1 = s, p_n = t$, 使得其任意前缀连通, 任意后缀连通。

- 一张图可以双极定向等价于存在一个点的排列 $p_1, p_2, \dots, p_n$, 其中 $p_1 = s, p_n = t$, 使得其任意前缀连通, 任意后缀连通。
- 证明略。

- 一张图可以双极定向等价于存在一个点的排列 $p_1, p_2, \dots, p_n$, 其中 $p_1 = s, p_n = t$, 使得其任意前缀连通, 任意后缀连通。
- 证明略。
- 满足该条件的排列的一种构造方式是双极定向后的拓扑序。

- 以下内容口胡严重，大家可以尝试感性理解。

构造双极定向

- 以下内容口胡严重，大家可以尝试感性理解。
- 朴素想法是直接耳分解然后暴力判断偏序关系，可惜这样是 $O(n^2)$ 的，不够优秀。

构造双极定向

- 以下内容口胡严重，大家可以尝试感性理解。
- 朴素想法是直接耳分解然后暴力判断偏序关系，可惜这样是 $O(n^2)$ 的，不够优秀。
- 考虑原问题可以缩二度点，不影响性质。

构造双极定向

- 以下内容口胡严重，大家可以尝试感性理解。
- 朴素想法是直接耳分解然后暴力判断偏序关系，可惜这样是 $O(n^2)$ 的，不够优秀。
- 考虑原问题可以缩二度点，不影响性质。
- 进一步地，求出 dfs 生成树后，我们只需要保留每个点子树内最浅的返祖边。

构造双极定向

- 以下内容口胡严重，大家可以尝试感性理解。
- 朴素想法是直接耳分解然后暴力判断偏序关系，可惜这样是 $O(n^2)$ 的，不够优秀。
- 考虑原问题可以缩二度点，不影响性质。
- 进一步地，求出 dfs 生成树后，我们只需要保留每个点子树内最浅的返祖边。
- 我们发现叶子一定是二度点，因此可以每次剥叶子，最后可以缩减成只剩 s 到 t 的链的平凡情况。

构造双极定向

- 以下内容口胡严重，大家可以尝试感性理解。
- 朴素想法是直接耳分解然后暴力判断偏序关系，可惜这样是 $O(n^2)$ 的，不够优秀。
- 考虑原问题可以缩二度点，不影响性质。
- 进一步地，求出 dfs 生成树后，我们只需要保留每个点子树内最浅的返祖边。
- 我们发现叶子一定是二度点，因此可以每次剥叶子，最后可以缩减成只剩 s 到 t 的链的平凡情况。
- 另一方面，我们考虑直接求出双极定向后的一个拓扑序，那么对于一个耳，当遍历到他的一个点时直接按这个方向把整个耳遍历一遍即可。

构造双极定向

- 以下内容口胡严重，大家可以尝试感性理解。
- 朴素想法是直接耳分解然后暴力判断偏序关系，可惜这样是 $O(n^2)$ 的，不够优秀。
- 考虑原问题可以缩二度点，不影响性质。
- 进一步地，求出 dfs 生成树后，我们只需要保留每个点子树内最浅的返祖边。
- 我们发现叶子一定是二度点，因此可以每次剥叶子，最后可以缩减成只剩 s 到 t 的链的平凡情况。
- 另一方面，我们考虑直接求出双极定向后的一个拓扑序，那么对于一个耳，当遍历到他的一个点时直接按这个方向把整个耳遍历一遍即可。
- 注意同时能遍历多个耳时要优先遍历较深的那个。

- 把上面两点结合一下，可以得到一种简单的实现方法：对于不在 s 到 t 的链上的点，按从下往上的顺序，将每个点加入其父亲和最浅返祖边的后继队列中。然后从 s 开始遍历链，每次走到一个点就 dfs 递归遍历他的后继队列中的点即可。

- 把上面两点结合一下，可以得到一种简单的实现方法：对于不在 s 到 t 的链上的点，按从下往上的顺序，将每个点加入其父亲和最浅返祖边的后继队列中。然后从 s 开始遍历链，每次走到一个点就 dfs 递归遍历他的后继队列中的点即可。
- 可以证明这个做法是对的。

- 给定一张图，初始时点均为白色。每次选择至多 k 个点染黑，使得每次操作后所有黑色点连通，所有白色点连通。
- 求最小的 k 并构造方案。
- $2 \leq n \leq 2 \times 10^5, 1 \leq m \leq 2.3 \times 10^5$ ，无重边无自环，保证图连通。

- 假设 $k = 1$ ，那就是双极定向。

- 假设 $k = 1$ ，那就是双极定向。
- 考虑什么时候 $k > 1$ ，当且仅当圆方树不是毛毛虫。

- 假设 $k = 1$ ，那就是双极定向。
- 考虑什么时候 $k > 1$ ，当且仅当圆方树不是毛毛虫。
- 对于第一个染黑的点和最后一个染黑的点 x, y ，显然它们一定是圆方树上的叶子，否则可以调整。

- 假设 $k = 1$ ，那就是双极定向。
- 考虑什么时候 $k > 1$ ，当且仅当圆方树不是毛毛虫。
- 对于第一个染黑的点和最后一个染黑的点 x, y ，显然它们一定是圆方树上的叶子，否则可以调整。
- 对于 x, y 间路径上某个圆点，与其相邻的不在路径上的子树内的点都要与他一起染黑。

- 假设 $k = 1$ ，那就是双极定向。
- 考虑什么时候 $k > 1$ ，当且仅当圆方树不是毛毛虫。
- 对于第一个染黑的点和最后一个染黑的点 x, y ，显然它们一定是圆方树上的叶子，否则可以调整。
- 对于 x, y 间路径上某个圆点，与其相邻的不在路径上的子树内的点都要与他一起染黑。
- 同理，对于路径上的黑点，与其相邻的不在路径上的子树，每个子树都要同时染黑。

- 假设 $k = 1$ ，那就是双极定向。
- 考虑什么时候 $k > 1$ ，当且仅当圆方树不是毛毛虫。
- 对于第一个染黑的点和最后一个染黑的点 x, y ，显然它们一定是圆方树上的叶子，否则可以调整。
- 对于 x, y 间路径上某个圆点，与其相邻的不在路径上的子树内的点都要与他一起染黑。
- 同理，对于路径上的黑点，与其相邻的不在路径上的子树，每个子树都要同时染黑。
- 找到所有需要同时染黑的连通块，最大的那个就是这对 x, y 的答案。

- 肯定不能暴力枚举 x, y , 于是考虑进一步观察性质。

- 肯定不能暴力枚举 x, y ，于是考虑进一步观察性质。
- 不妨以某个方点为根，然后找到 x, y 的 LCA z 。

- 肯定不能暴力枚举 x, y ，于是考虑进一步观察性质。
- 不妨以某个方点为根，然后找到 x, y 的 LCA z 。
- 对于路径 $z \rightarrow x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_k = x$ ，可以发现 $\forall 0 \leq i < k$ ， x_{i+1} 一定是 x_i 最大的那个儿子。

- 肯定不能暴力枚举 x, y ，于是考虑进一步观察性质。
- 不妨以某个方点为根，然后找到 x, y 的 LCA z 。
- 对于路径 $z \rightarrow x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_k = x$ ，可以发现 $\forall 0 \leq i < k$ ， x_{i+1} 一定是 x_i 最大的那个儿子。
- 证明考虑如果不是的话调整过来答案不劣。

- 肯定不能暴力枚举 x, y ，于是考虑进一步观察性质。
- 不妨以某个方点为根，然后找到 x, y 的 LCA z 。
- 对于路径 $z \rightarrow x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_k = x$ ，可以发现 $\forall 0 \leq i < k$ ， x_{i+1} 一定是 x_i 最大的那个儿子。
- 证明考虑如果不是的话调整过来答案不劣。
- y 那边同理。

- 肯定不能暴力枚举 x, y ，于是考虑进一步观察性质。
- 不妨以某个方点为根，然后找到 x, y 的 LCA z 。
- 对于路径 $z \rightarrow x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_k = x$ ，可以发现 $\forall 0 \leq i < k$ ， x_{i+1} 一定是 x_i 最大的那个儿子。
- 证明考虑如果不是的话调整过来答案不劣。
- y 那边同理。
- 同理也可以得出 x_0, y_0 一定是 z 最大的两个儿子。

- 肯定不能暴力枚举 x, y ，于是考虑进一步观察性质。
- 不妨以某个方点为根，然后找到 x, y 的 LCA z 。
- 对于路径 $z \rightarrow x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_k = x$ ，可以发现 $\forall 0 \leq i < k$ ， x_{i+1} 一定是 x_i 最大的那个儿子。
- 证明考虑如果不是的话调整过来答案不劣。
- y 那边同理。
- 同理也可以得出 x_0, y_0 一定是 z 最大的两个儿子。
- 简单树形 DP 即可求出答案。同时也求出了对应的 x 和 y 。

- 肯定不能暴力枚举 x, y ，于是考虑进一步观察性质。
- 不妨以某个方点为根，然后找到 x, y 的 LCA z 。
- 对于路径 $z \rightarrow x_0 \rightarrow x_1 \rightarrow \cdots \rightarrow x_k = x$ ，可以发现 $\forall 0 \leq i < k$ ， x_{i+1} 一定是 x_i 最大的那个儿子。
- 证明考虑如果不是的话调整过来答案不劣。
- y 那边同理。
- 同理也可以得出 x_0, y_0 一定是 z 最大的两个儿子。
- 简单树形 DP 即可求出答案。同时也求出了对应的 x 和 y 。
- 只保留路径上经过的点双然后跑双极定向即可。

广义串并联图

- 广义串并联图是指不存在同胚于 K_4 的子图的图。

- 广义串并联图是指不存在同胚于 K_4 的子图的图。
- 能通过删一度点、缩二度点、叠合重边变成一个单点。

- 广义串并联图是指不存在同胚于 K_4 的子图的图。
- 能通过删一度点、缩二度点、叠合重边变成一个单点。
- 去掉重边后, $|E| \leq 2|V|$ 。

- 广义串并联图是指不存在同胚于 K_4 的子图的图。
- 能通过删一度点、缩二度点、叠合重边变成一个单点。
- 去掉重边后, $|E| \leq 2|V|$ 。
- 任何一个广义串并联图都可以按以下方式构造:

- 广义串并联图是指不存在同胚于 K_4 的子图的图。
- 能通过删一度点、缩二度点、叠合重边变成一个单点。
- 去掉重边后, $|E| \leq 2|V|$ 。
- 任何一个广义串并联图都可以按以下方式构造:
 - 初始时只有一个单点。

- 广义串并联图是指不存在同胚于 K_4 的子图的图。
- 能通过删一度点、缩二度点、叠合重边变成一个单点。
- 去掉重边后, $|E| \leq 2|V|$ 。
- 任何一个广义串并联图都可以按以下方式构造:
 - 初始时只有一个单点。
 - 每次增加一个点, 并和原图某个点连一条边。

- 广义串并联图是指不存在同胚于 K_4 的子图的图。
- 能通过删一度点、缩二度点、叠合重边变成一个单点。
- 去掉重边后, $|E| \leq 2|V|$ 。
- 任何一个广义串并联图都可以按以下方式构造:
 - 初始时只有一个单点。
 - 每次增加一个点, 并和原图某个点连一条边。
 - 选择两个直接相连的点, 新加一个点与这两个点相连, 并选择是否删去原本的那条边。

- 指通过“删一度点、缩二度点、叠合重边”这三个操作。

- 指通过“删一度点、缩二度点、叠合重边”这三个操作。
- 一般用到这个方法的题的常见特征是边数与点数的差很小。

- 指通过“删一度点、缩二度点、叠合重边”这三个操作。
- 一般用到这个方法的题的常见特征是边数与点数的差很小。
- 原因是在以上操作过程中， $k = |E| - |V|$ 单调不增，并且缩合后每个点的度数都至少为 3，因此 $2|E| \geq 3|V|$ ，因此最后的图满足 $|V| \leq 2k, |E| \leq 3k$ 。

- 求给 n 个点 m 条边的无向图定向, 使得其变为 DAG 的方案数。
- $n \leq 10^5, m - n \leq 10$ 。

- 考虑用广义串并联图缩小问题规模。

- 考虑用广义串并联图缩小问题规模。
- 考虑缩二度点的时候，如果两条边定向相同相当于变成一条边，不同相当于把这条边断掉。

- 考虑用广义串并联图缩小问题规模。
- 考虑缩二度点的时候，如果两条边定向相同相当于变成一条边，不同相当于把这条边断掉。
- 因此用一个二元组 (f, g) 来表示一条边的贡献，其中 f 是定向的贡献， g 是断开的贡献。

- 考虑用广义串并联图缩小问题规模。
- 考虑缩二度点的时候，如果两条边定向相同相当于变成一条边，不同相当于把这条边断掉。
- 因此用一个二元组 (f, g) 来表示一条边的贡献，其中 f 是定向的贡献， g 是断开的贡献。
- 删一度点：直接将 $2f + g$ 乘入答案。

- 考虑用广义串并联图缩小问题规模。
- 考虑缩二度点的时候，如果两条边定向相同相当于变成一条边，不同相当于把这条边断掉。
- 因此用一个二元组 (f, g) 来表示一条边的贡献，其中 f 是定向的贡献， g 是断开的贡献。
- 删一度点：直接将 $2f + g$ 乘入答案。
- 缩二度点： $f' = f_1 f_2, g' = g_1 g_2 + 2f_1 g_2 + 2f_2 g_1 + 2f_1 f_2$ 。

- 考虑用广义串并联图缩小问题规模。
- 考虑缩二度点的时候，如果两条边定向相同相当于变成一条边，不同相当于把这条边断掉。
- 因此用一个二元组 (f, g) 来表示一条边的贡献，其中 f 是定向的贡献， g 是断开的贡献。
- 删一度点：直接将 $2f + g$ 乘入答案。
- 缩二度点： $f' = f_1 f_2, g' = g_1 g_2 + 2f_1 g_2 + 2f_2 g_1 + 2f_1 f_2$ 。
- 叠合重边： $f' = f_1 f_2 + f_1 g_2 + f_2 g_1, g' = g_1 g_2$ 。

- 缩完后有 $n \leq 20$ ，是典。

- 缩完后有 $n \leq 20$ ，是典。
- 设 f_S 为集合 S 的答案，则：

$$f_S = \sum_{T \subseteq S, T \neq \emptyset} f(S \setminus T) w_1(T) w_2(T, S)$$

- 缩完后有 $n \leq 20$ ，是典。
- 设 f_S 为集合 S 的答案，则：

$$f_S = \sum_{T \subseteq S, T \neq \emptyset} f(S \setminus T) w_1(T) w_2(T, S)$$

- 其中 $w_1(T)$ 是 T 中每条边都断开的方案数即 $\prod g$ ， $w_2(T, S)$ 是 T 与 S 间每条边都从 T 连向 S 的贡献，即 $\prod (f + g)$ 。

- 缩完后有 $n \leq 20$ ，是典。
- 设 f_S 为集合 S 的答案，则：

$$f_S = \sum_{T \subseteq S, T \neq \emptyset} f(S \setminus T) w_1(T) w_2(T, S)$$

- 其中 $w_1(T)$ 是 T 中每条边都断开的方案数即 $\prod g$ ， $w_2(T, S)$ 是 T 与 S 间每条边都从 T 连向 S 的贡献，即 $\prod (f + g)$ 。
- 实现时可以把系数写成 $\prod_{(u_i, v_i) \in T} \frac{g_i}{f_i + g_i}$ ，最后乘上所有边的 $(f + g)$ 的乘积即可。

- 给你一张 n 个点 m 条边的连通无向图，边有边权，求是否存在一条从 1 到 n 的路径使得路径长度大于最短路。
- $2 \leq n \leq 10^5, 1 \leq m \leq 2 \times 10^5$ ，边权范围为 $[1, 10^9] \cap \mathbb{N}$ 。

- 结论：首先对于一个点双连通分量，如果存在同胚于 K_4 的子图，则一定有解。

- 结论：首先对于一个点双连通分量，如果存在同胚于 K_4 的子图，则一定有解。
- 证明：设这四个点为 a, b, c, d ，考虑反证。

- 结论：首先对于一个点双连通分量，如果存在同胚于 K_4 的子图，则一定有解。
- 证明：设这四个点为 a, b, c, d ，考虑反证。
-

$$w(a, b) + w(b, d) = w(a, c) + w(c, b) + w(b, d)$$

$$\implies w(a, b) = w(a, c) + w(c, b)$$

$$w(a, c) + w(c, d) = w(a, b) + w(b, c) + w(c, d)$$

$$\implies w(a, c) = w(a, b) + w(b, c)$$

- 结论：首先对于一个点双连通分量，如果存在同胚于 K_4 的子图，则一定有解。
- 证明：设这四个点为 a, b, c, d ，考虑反证。

-

$$w(a, b) + w(b, d) = w(a, c) + w(c, b) + w(b, d)$$

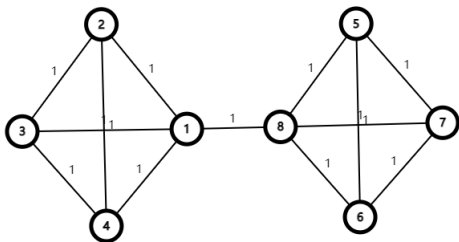
$$\implies w(a, b) = w(a, c) + w(c, b)$$

$$w(a, c) + w(c, d) = w(a, b) + w(b, c) + w(c, d)$$

$$\implies w(a, c) = w(a, b) + w(b, c)$$

- 可以得出 $w(b, c) = w(a, b) - w(a, c) = w(a, c) - w(a, b) = 0$ 。

- 但是这个结论对一般图不成立，考虑如下 hack (by Froggy):



洛谷

- 那是不是只要保留 1 到 n 路径上的点双就好了？

- 那是不是只要保留 1 到 n 路径上的点双就好了？
- 还真是。

- 那是不是只要保留 1 到 n 路径上的点双就好了？
- 还真是。
- 具体来说就是在 1 和 n 之间连一条边权为最短路的边，然后只考虑包含 1, n 的那个点双就好了。

- 那是不是只要保留 1 到 n 路径上的点双就好了？
- 还真是。
- 具体来说就是在 1 和 n 之间连一条边权为最短路边，然后只考虑包含 1, n 的那个点双就好了。
- 然后考虑怎么 check。

- 那是不是只要保留 1 到 n 路径上的点双就好了？
- 还真是。
- 具体来说就是在 1 和 n 之间连一条边权为最短路边，然后只考虑包含 1, n 的那个点双就好了。
- 然后考虑怎么 check。
- 对于一度点，直接扔掉。

- 那是不是只要保留 1 到 n 路径上的点双就好了？
- 还真是。
- 具体来说就是在 1 和 n 之间连一条边权为最短路边，然后只考虑包含 1, n 的那个点双就好了。
- 然后考虑怎么 check。
- 对于一度点，直接扔掉。
- 对于二度点，新边的权值为原来两条边边权之和。

- 那是不是只要保留 1 到 n 路径上的点双就好了？
- 还真是。
- 具体来说就是在 1 和 n 之间连一条边权为最短路的边，然后只考虑包含 1, n 的那个点双就好了。
- 然后考虑怎么 check。
- 对于一度点，直接扔掉。
- 对于二度点，新边的权值为原来两条边边权之和。
- 对于重边，如果边权不同，则将边权标记为 -1 表示这条边不能走。

- 那是不是只要保留 1 到 n 路径上的点双就好了？
- 还真是。
- 具体来说就是在 1 和 n 之间连一条边权为最短路的边，然后只考虑包含 1, n 的那个点双就好了。
- 然后考虑怎么 check。
- 对于一度点，直接扔掉。
- 对于二度点，新边的权值为原来两条边边权之和。
- 对于重边，如果边权不同，则将边权标记为 -1 表示这条边不能走。
- 注意 1 和 n 不能缩。

- 那是不是只要保留 1 到 n 路径上的点双就好了？
- 还真是。
- 具体来说就是在 1 和 n 之间连一条边权为最短路边，然后只考虑包含 1, n 的那个点双就好了。
- 然后考虑怎么 check。
- 对于一度点，直接扔掉。
- 对于二度点，新边的权值为原来两条边边权之和。
- 对于重边，如果边权不同，则将边权标记为 -1 表示这条边不能走。
- 注意 1 和 n 不能缩。
- 最后如果只剩下 1 和 n 且中间的边边权不为 -1 则无解，否则有解。

- 终于写完辣!

